

B.M.S. COLLEGE OF ENGINEERING BENGALURU

Autonomous Institute, Affiliated to VTU



Lab Record

Object-Oriented Modeling – 23CS5PCOOM

Submitted in partial fulfillment for the 5th Semester Laboratory

Bachelor of Engineering
in
Computer Science and Engineering

Submitted by:

Mechineni Anjutharuni

1BM23CS187

Department of Computer Science and Engineering
B.M.S. College of Engineering
Bull Temple Road, Basavanagudi, Bangalore 560 019
August 2025-December 2025

B.M.S. COLLEGE OF ENGINEERING
DEPARTMENT OF COMPUTER SCIENCE AND ENGINEERING



CERTIFICATE

This is to certify that the Object-Oriented Modeling(23CS5PCOOM) laboratory has been carried out by Mechineni Anjutharuni (1BM23CS187) during the 5th Semester August 2025-December 2025

Signature of the Faculty Incharge:
Sheetal V A (Assistant Professor)

Department of Computer Science and Engineering
B.M.S. College of Engineering, Bangalore

Table of Contents

1. Hotel Management System
2. Credit Card Processing
3. Library Management System
4. Stock Maintenance System
5. Passport Automation System

1. Hotel Management System

Problem Statement

Hotels often face challenges in managing bookings, check-ins, check-outs, and billing when done manually. This results in errors like double booking, loss of records, and customer dissatisfaction. The problem is the absence of an efficient, automated system that can handle hotel operations smoothly.

SRS-Software Requirements Specification

1. Introduction

1.1 Purpose of this Document

The purpose of this SRS document is to describe the Hotel Management System in detail. It defines the system's objectives, requirements, and functionality, which will help the development team and also act as a reference for testing.

1.2 Scope of this Document

The Hotel Management System will automate tasks such as reservations, customer check-in/check-out, staff management, billing, and reporting. This reduces manual work, increases efficiency, and ensures customer satisfaction.

1.3 Overview

The HMS is a user-friendly application that handles hotel activities. It consists of modules for room booking, billing, customer records, and reporting. The system reduces errors and increases efficiency for hotel management.

2. General Description

The HMS is meant for receptionists, hotel managers, and administrators. It provides features such as viewing available rooms, managing staff, handling payments, and generating reports. The objective is to reduce paperwork and improve accuracy.

3. Functional Requirements

1. Customer Registration
2. Room Booking and Cancellation
3. Check-in and Check-out Processing
4. Billing and Payment Management
5. Staff Management
6. Report Generation (daily, monthly, yearly)

4. Interface Requirements

The system will provide a GUI for users with options like booking, billing, and staff management. It will also include a backend database (MySQL) for storing records.

5. Performance Requirements

The HMS should process bookings quickly and allow multiple users at the same time. It should handle up to 5000 records efficiently, and responses should be under 2 seconds.

6. Design Constraints

1. Developed using Java/Python and MySQL.
2. Should meet data security standards for customer details.
3. Should run on Windows and Linux OS.

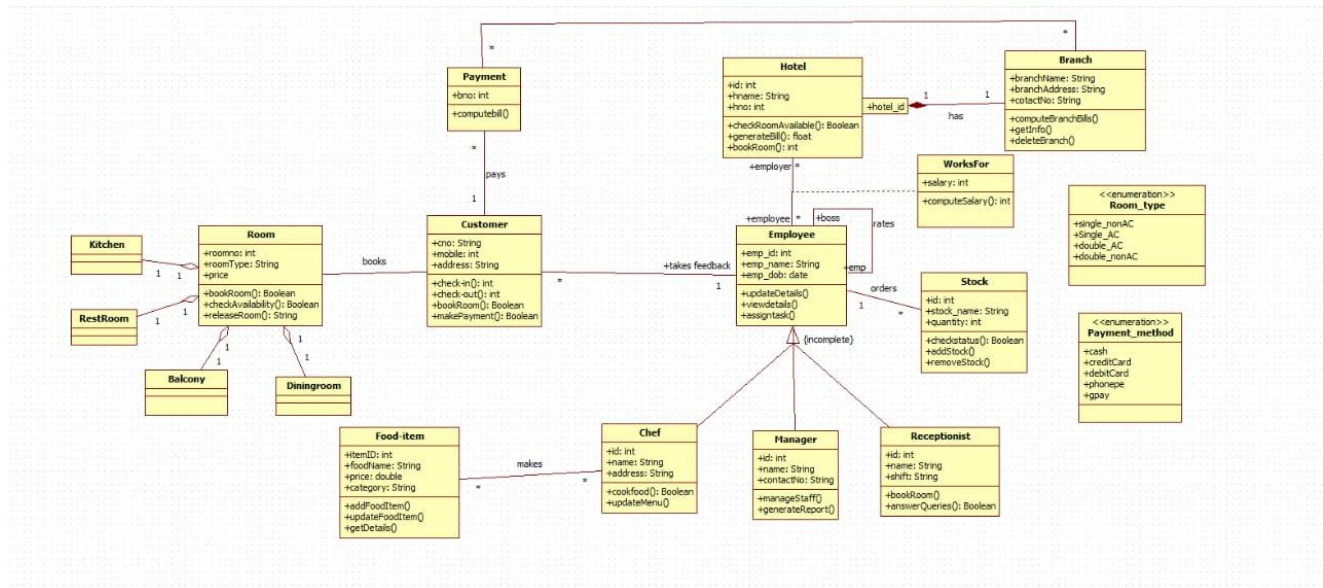
7. Non-Functional Attributes

1. Security: Protect customer data with encryption.
2. Reliability: Should be available 24/7 with 99% uptime.
3. Scalability: Must support hotel expansion.
4. Usability: Easy for staff to use.
5. Portability: Compatible with different platforms.

8. Preliminary Schedule and Budget

Development is estimated to take 4 months with a budget of \$20,000. This includes design, coding, testing, and deployment.

Class Diagram

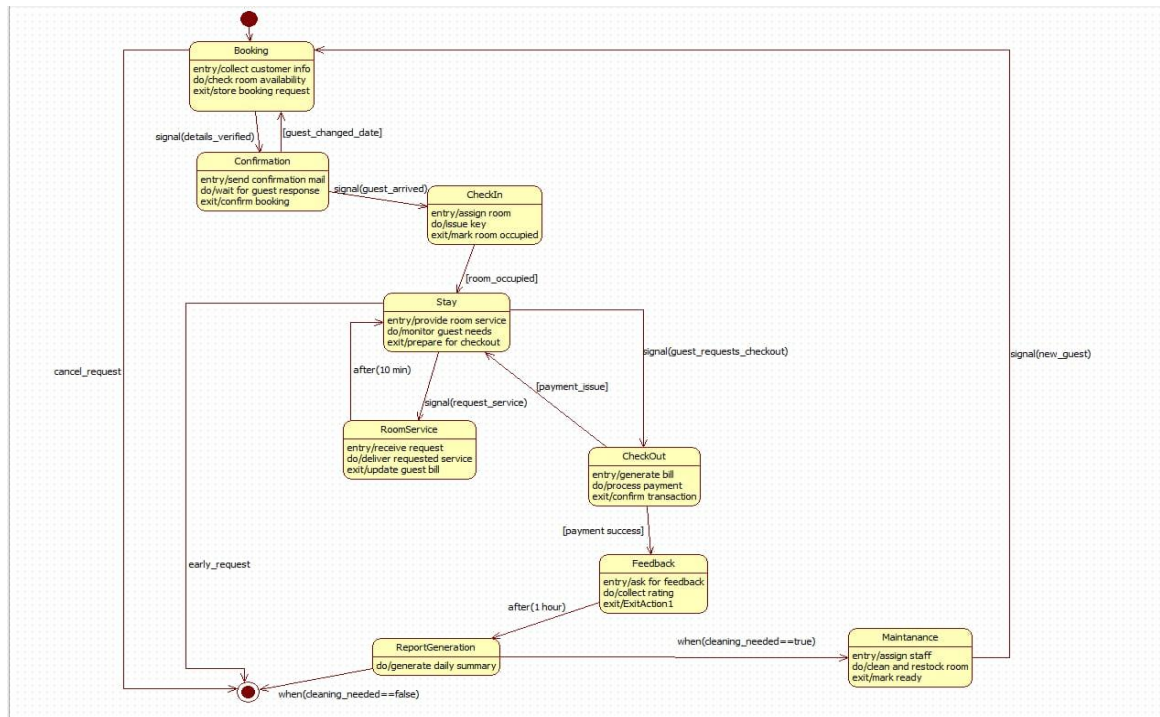


The class diagram shows the structure of a Hotel Management System. It includes major classes such as Hotel, Customer, Employee, Room, Payment, FoodItem, Stock, and Branch.

- A Customer books rooms, orders food, and makes payments.
- **Hotel** manages rooms, employees, and billing.
- **Employees** (Chef, Manager, Receptionist) work for the hotel and perform specific tasks.
- **Rooms** consist of Bedroom, Kitchen, and Bathroom and have different types defined by an enumeration.
- **FoodItem** represents items prepared by chefs and ordered by customers.
- **Stock** handles inventory management.
- **Payment** processes bill calculations using different payment modes.
- **Branch** represents different hotel branches with revenue and profit calculations.

Overall, the diagram depicts relationships between customers, hotel operations, employees, rooms, services, and payments.

Simple State Diagram:

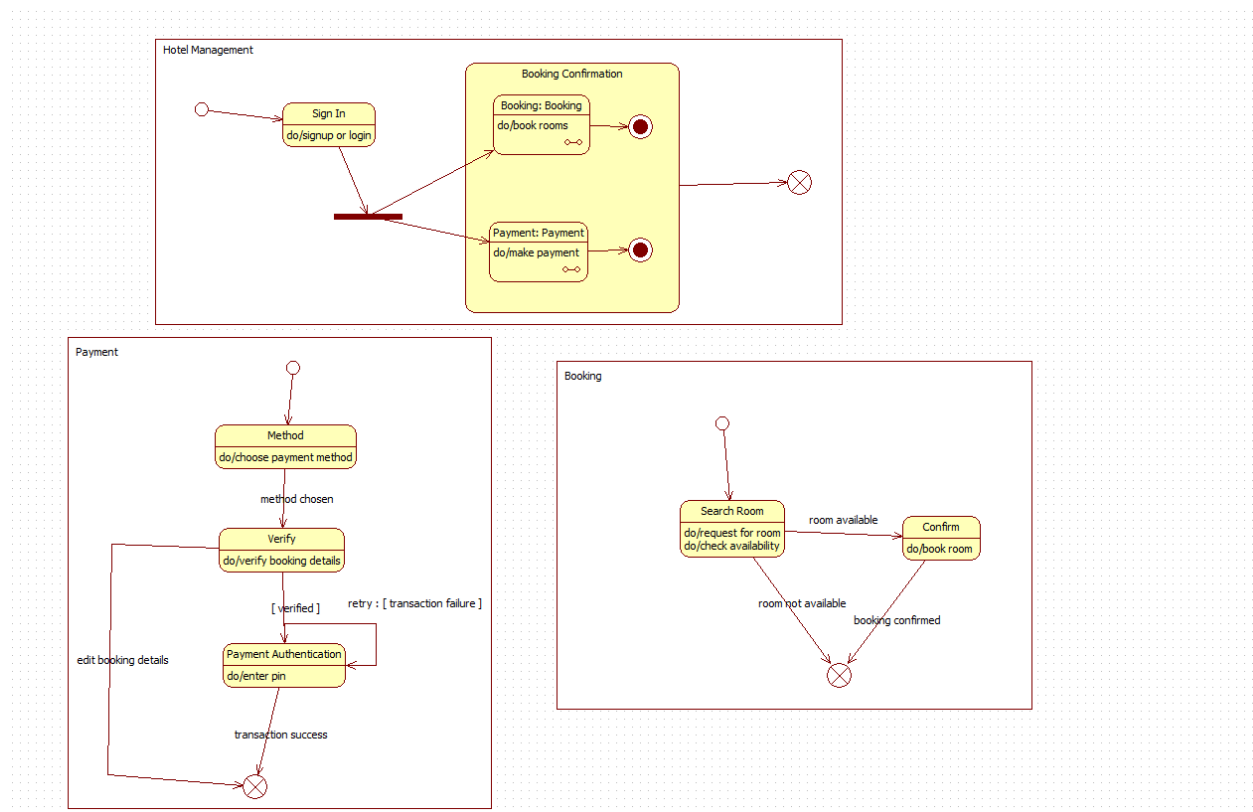


This state machine models the guest journey in a Hotel Management System. It starts with **Booking**, where customer details are collected and room availability is checked. After verification, the system moves to **Confirmation** and then to **CheckIn** once the guest arrives.

The workflow enters the **Stay** composite state, which manages room occupation, room service requests, and checkout preparation. When the guest requests checkout, the process moves to **CheckOut** for bill generation and payment. After successful payment, the system collects guest **Feedback** and then generates a **Report**.

Based on conditions, the system may transition to **Maintenance** for cleaning and restocking before reaching the final state. Signals and guard conditions control transitions throughout the process.

Advanced State Diagram:



This state machine diagram represents the Hotel Management System workflow using composite states and submachine states for Booking and Payment processes.

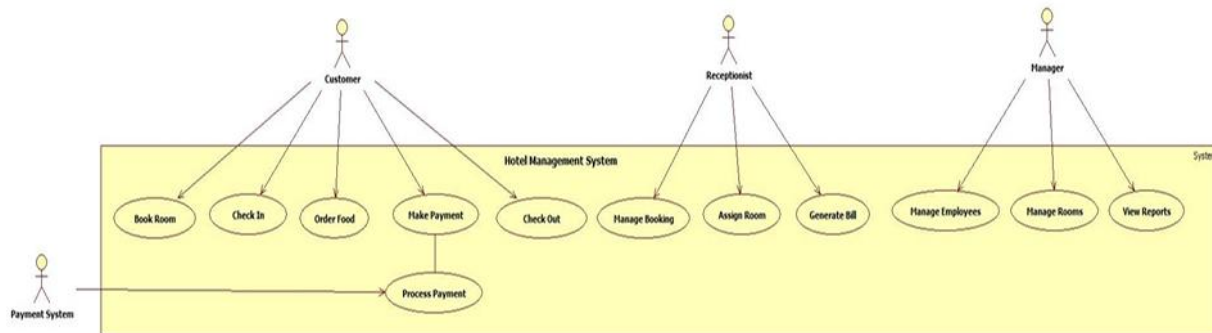
The system starts at the initial pseudostate, transitions to Sign In, and then enters the Booking Confirmation composite state, which contains two independent submachine states:

- Booking submachine – handles room search and confirmation using guard conditions (room available / not available).
- Payment submachine – manages payment method selection, verification, authentication, and retry loops for transaction failure.

Each submachine includes its own entry and exit points, internal transitions, and final states, representing completion of that process.

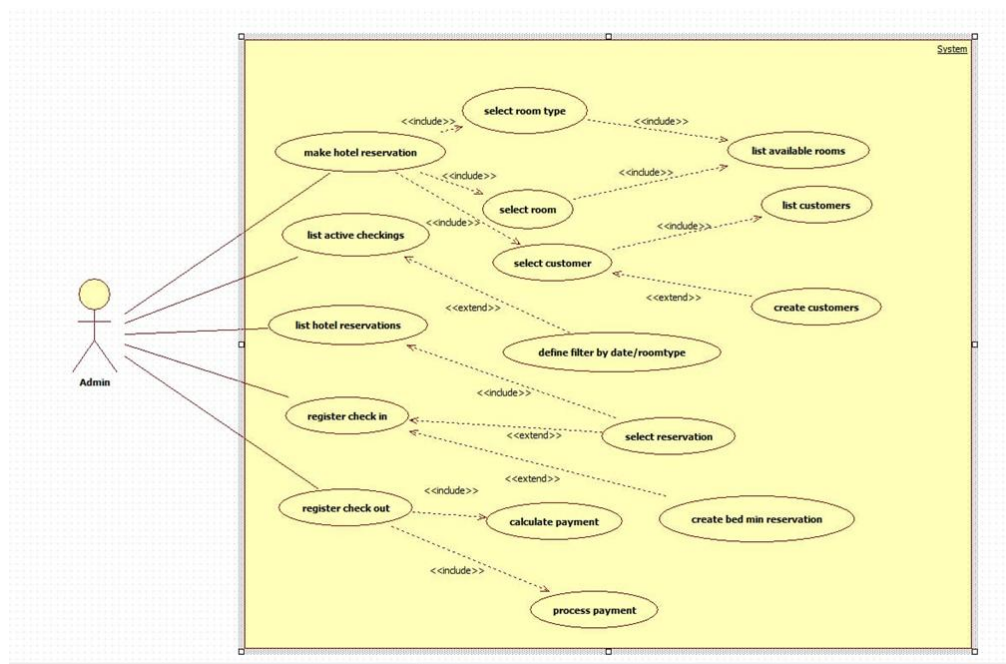
Once both submachines complete, the system reaches the final pseudostate, completing the booking-and-payment workflow.

Simple Use-Case Diagram



The simple use case diagram shows the basic everyday functions of the Hotel Management System. It involves four main actors: Customer, Receptionist, Manager, and the external Payment System. The Customer can perform common tasks like booking a room, checking in, ordering food, making payments, and checking out. The Receptionist handles bookings, assigns rooms, and prepares bills. The Manager has administrative responsibilities such as managing employees, handling rooms, and viewing reports. The Payment System helps process customer payments. Overall, this diagram focuses only on the essential hotel operations and is easy to understand because it highlights basic interactions between users and the system.

Advanced Use case Diagram



This use case diagram represents the Admin's interaction with the Hotel Management System. The Admin actor can perform major operations such as making reservations, listing active check-ins, listing reservations, and registering check-in/check-out.

The main use cases include multiple <<include>> and <<extend>> relationships to show reusable and optional behaviors:

- *Make hotel reservation* includes selecting room type, listing available rooms, selecting room, and selecting customer. It may also extend to creating a new customer.
- *List hotel reservations* can be extended by applying filters such as date or room type.
- *Register check-in* and *register check-out* extend the flow by selecting a reservation and calculating payment.
- Payment processing is included as part of the checkout workflow.

Overall, the diagram demonstrates how the Admin manages bookings, customer details, room allocation, and payment operations using interconnected and reusable use cases.

Scenarios

1: Book Room

The customer opens the hotel booking page.

The customer selects the desired room type (Single, Double, AC, Non-AC, etc.).

The system filters and displays all available rooms of that type.

The customer chooses one room from the list.

The customer enters personal details (name, contact number, ID proof).

The system verifies the entered information.

The customer confirms the booking.

The system generates a booking confirmation and stores reservation details.

2:Process Payment

The customer selects the “Make Payment” option. The system displays the total bill amount.

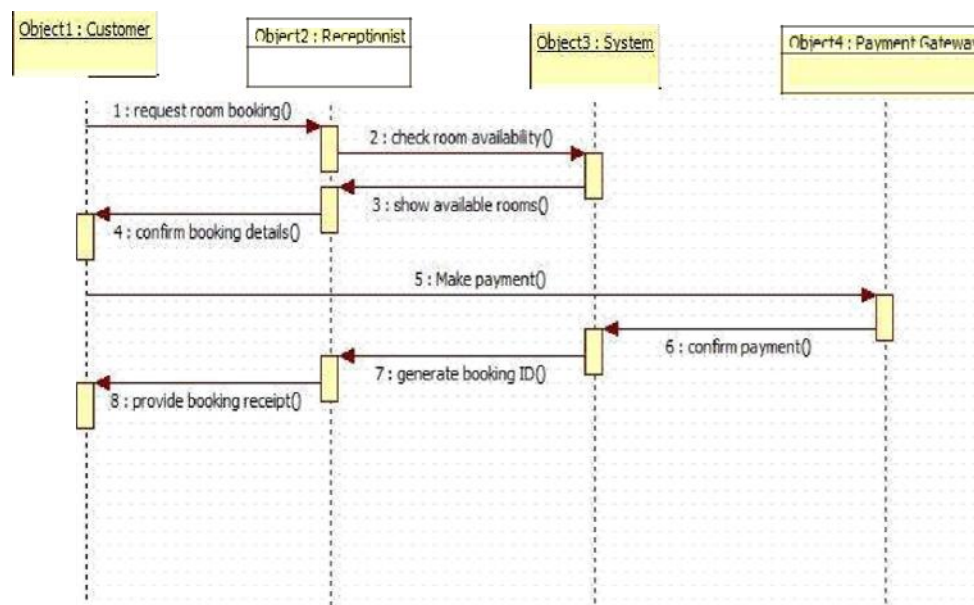
The customer chooses a payment method (UPI, card, banking, etc.). The system sends the payment request to the payment gateway.

The customer enters payment details.

The payment gateway validates the details and processes the transaction. On success, the system updates the payment status.

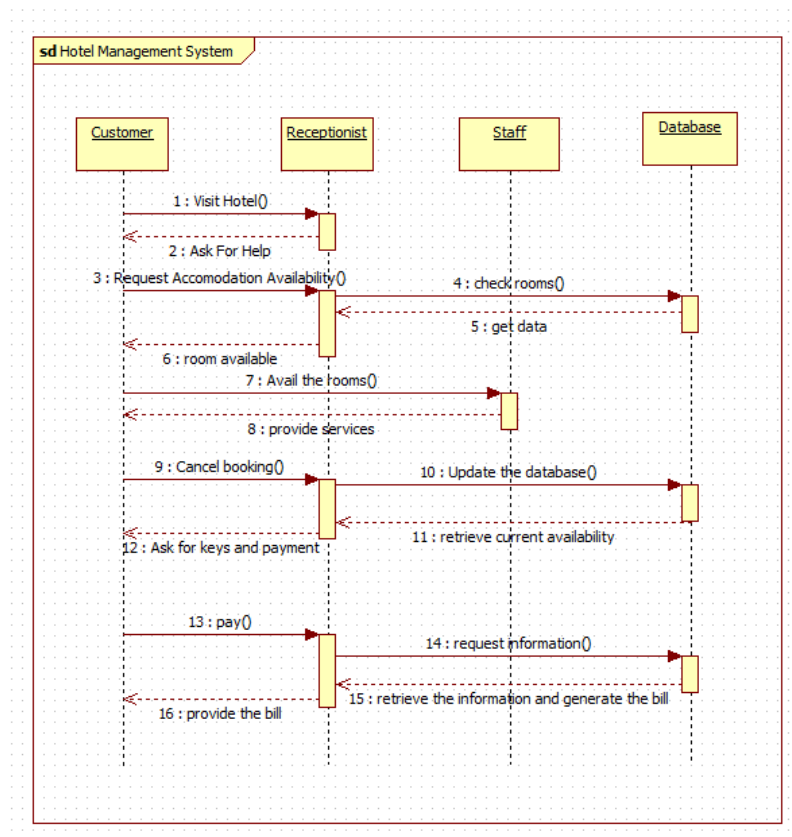
A receipt is generated and shown to the customer.

Simple sequence diagram



The simple sequence diagram explains the basic steps involved in booking a room. The process begins when the Customer asks the Receptionist to book a room. The receptionist checks availability by sending a request to the System, which returns a list of available rooms. After seeing the options, the customer confirms the booking. Next, the customer proceeds to make the payment. The receptionist sends the payment request to the Payment Gateway, which verifies the transaction and returns the result. If the payment is successful, the system creates a booking ID, and the receptionist gives the customer a booking receipt. This diagram focuses only on the key tasks—checking rooms, confirming booking, and processing payment—without showing extra steps or internal system details.

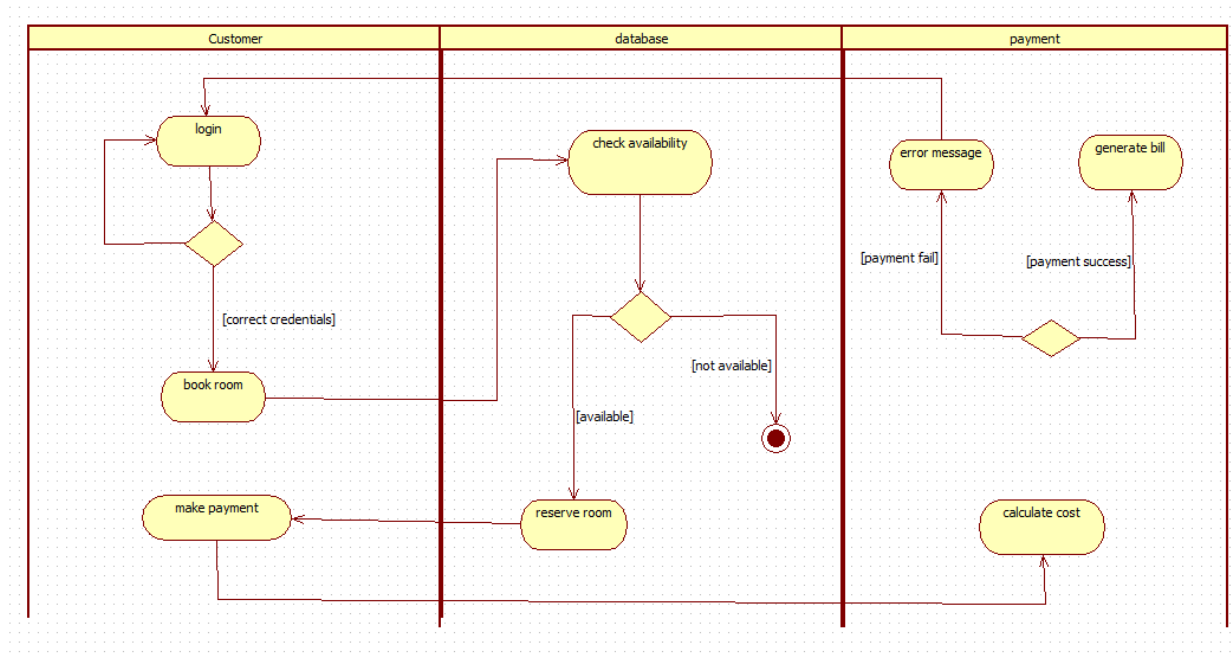
Advanced Sequence Diagram:



This sequence diagram illustrates the interaction between the Customer, Receptionist, Staff, and Database during hotel booking and service processes. The flow begins with the customer requesting room availability. The receptionist forwards this request to staff, who fetch room data from the database and return availability status.

Once rooms are available, the customer avails them, and staff provide services. If the customer cancels, the receptionist updates the database with the latest availability. At checkout, the customer requests keys and billing. The receptionist initiates a payment request and retrieves necessary information from the database. After payment, the system generates and provides the final bill to the customer.

Activity diagram



This activity diagram represents the end-to-end hotel room booking workflow across three swimlanes: Customer, Database, and Payment.

The process begins with the customer attempting to log in. Based on the decision node, incorrect credentials loop back to retry, while correct credentials allow the customer to proceed to book a room. The system then interacts with the Database to check room availability.

If rooms are available, the database proceeds to reserve the room, enabling the customer to make payment. The payment lane calculates cost and evaluates the transaction. A decision node directs the flow to either an error message (on failure) or generate bill (on success), which leads to the final activity.

If rooms are not available, the process terminates early through a final node.

2.Credit Card Processing

Problem Statement

In today's world, most payments are done using credit cards. However, manual processing of credit card payments causes delays, errors, and sometimes fraud. Customers face problems like declined transactions even when they have sufficient balance, while banks and merchants face the risk of fraudulent activities. There is a need for a software-based system that can process credit card transactions automatically, securely, and quickly to improve customer experience and trust.

SRS-Software Requirements Specification

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to describe the Credit Card Processing System in detail. It explains the objectives, features, and functionalities that the system should provide. This document will act as a guideline for developers, testers, and stakeholders to design and implement the system correctly.

1.2 Scope of this Document

The system will automate the entire process of credit card transactions such as customer authentication, authorization, billing, and fraud detection. It will improve the speed and reliability of transactions, reduce manual work, and provide security against fraud. This system will be used by banks, merchants, and customers.

1.3 Overview

The Credit Card Processing System will allow merchants to process payments securely in real-time. It will include modules for customer authentication, authorization, billing, fraud detection, and reporting. The main goal of the system is to ensure accurate, fast, and secure handling of transactions.

2. General Description

The system will serve customers, merchants, and banks. Customers will use their credit card for payments, merchants will initiate the transaction, and the bank will authorize and settle it. The system will ensure quick verification, secure payment, and record maintenance. It will also provide reporting facilities for auditing and analysis.

3. Functional Requirements

1. Customer Authentication (PIN/OTP)
2. Transaction Authorization (check credit balance, validity)
3. Fraud Detection and Prevention (suspicious activity check)
4. Billing and Settlement between bank and merchant
5. Transaction History Management
6. Report Generation for banks and merchants

4. Interface Requirements

The system will provide a secure web interface for merchants and administrators. Customers will authenticate using OTPs or PINs during transactions. The system will also integrate with banking servers through secure APIs for authorization and settlement.

5. Performance Requirements

- The system should handle thousands of transactions per second.
- Response time for authorization should be less than 1 second.
- The system should ensure 99.9% uptime for continuous service.

6. Design Constraints

1. The system must follow **PCI-DSS** security standards.
2. Encryption methods such as **AES** and **RSA** must be used.
3. The system must integrate with different banks' existing servers.

7. Non-Functional Attributes

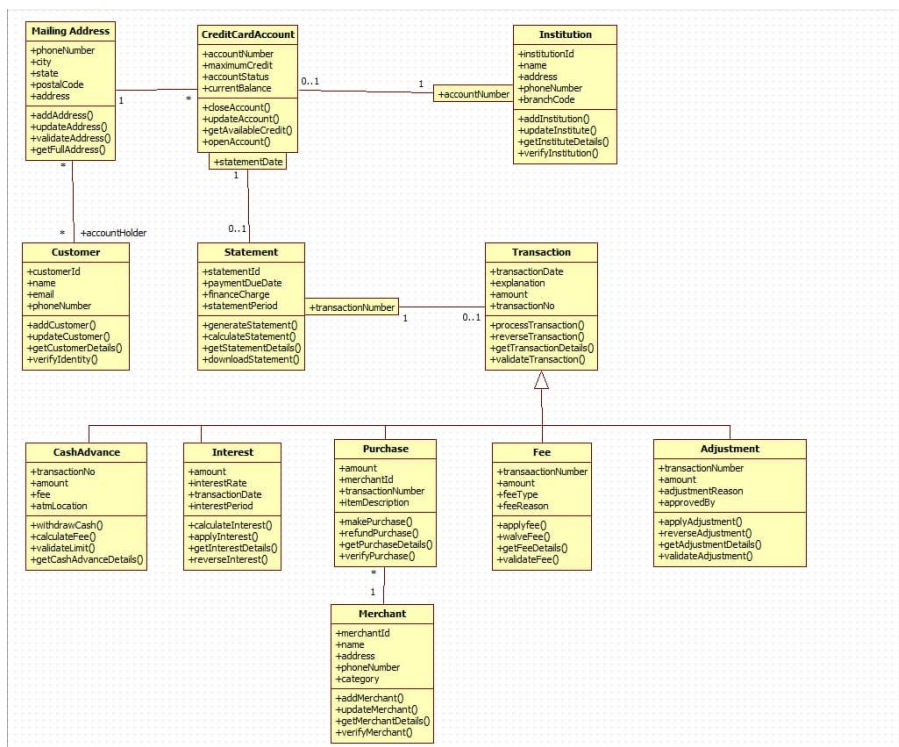
1. **Security:** Transactions must be encrypted and safe.
2. **Reliability:** System must not fail during high transaction loads.

3. **Scalability:** Should support increasing number of users and merchants.
4. **Usability:** Interface should be simple for merchants and banks.
5. **Compatibility:** Must work with different banking and merchant systems.

8. Preliminary Schedule and Budget

The development of the system is expected to take **6 months** with an estimated budget of **\$50,000**. This includes requirements gathering, design, coding, security testing, and deployment.

Class Diagram



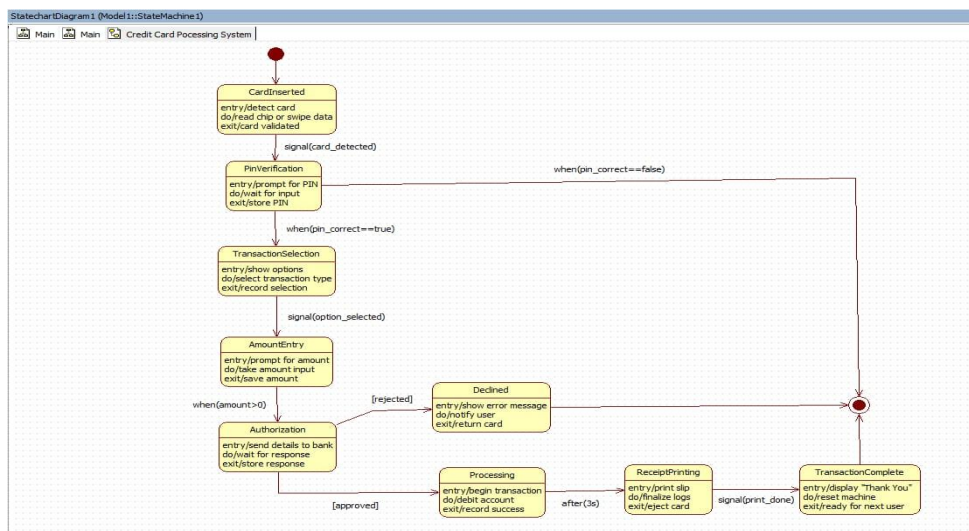
This class diagram represents a Credit Card Management System, modeling the relationships between customers, their credit card accounts, statements, transactions, and institutions. A Customer maintains one or more Mailing Addresses and holds a CreditCardAccount issued by an Institution. Each account generates periodic Statements, which contain multiple Transactions.

The Transaction class is a general superclass, specialized into CashAdvance, Interest, Purchase, Fee, and Adjustment, each defining additional attributes and transaction-specific operations such

as `applyFee()`, `reverseInterest()`, `makePurchase()`, or `applyAdjustment()`. Purchases are linked to a Merchant, which stores merchant identity and contact details.

The system supports operations like processing transactions, generating statements, validating accounts, updating institution records, and maintaining complete customer identity and address information. Overall, the diagram shows inheritance, associations, and one-to-many relationships to manage the full lifecycle of credit card usage—from account creation and merchant purchases to interest calculation, fees, adjustments, and monthly statements.

Simple state diagram



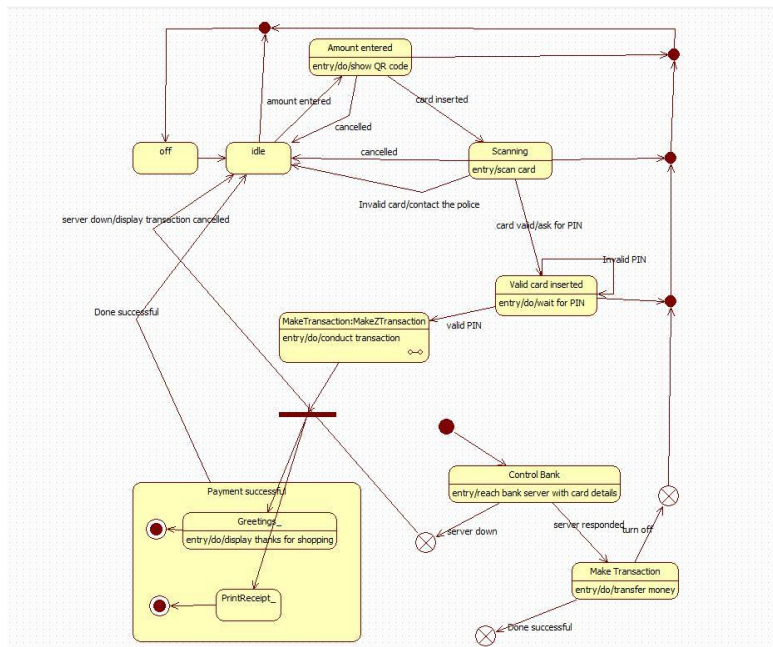
This state machine diagram represents the credit card transaction flow in a Credit Card Processing System. The process begins in the Card Inserted state, where the system reads and validates card data. It then transitions to PIN Verification, prompting the user for a PIN. A guard condition checks the PIN: if incorrect, the flow terminates; if correct, the system proceeds to Transaction Selection.

After the user selects the type of transaction, the system moves to Amount Entry, where the user inputs a transaction amount. If the amount is zero or invalid, the request is Declined. For valid amounts, the system enters Authorization, sending details to the bank and waiting for approval.

If the bank rejects the request, the system transitions to Declined and ejects the card. If approved, the flow moves to Processing, where the transaction is executed and account balances are updated. Once processed, the system enters Receipt Printing, printing the slip and confirming completion.

The final state, Transaction Complete, displays a thank-you message and resets the system for the next user.

Advance state diagram

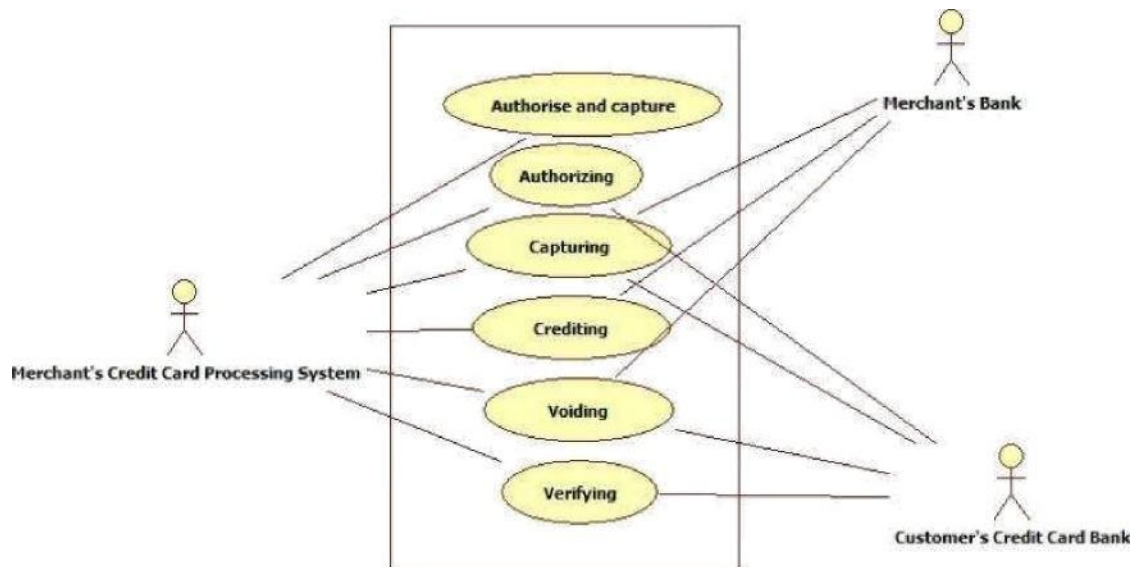


This state machine diagram models the workflow of a Digital Payment / POS Terminal System. The system begins in the Idle state, where the user can either enter an amount (triggering QR code generation) or insert a card. On card insertion, the system transitions to Scanning, validating the card. Invalid cards lead to termination, while valid cards move to the Valid Card Inserted state for PIN verification. An incorrect PIN cancels the transaction, while a valid PIN sends the system into Make Transaction, where the transaction is executed.

The system communicates with the Control Bank, which processes card details and authorizes the transaction. If the server is down or the bank declines, the flow terminates. Successful authorization leads to the Payment Successful composite state, which includes Greeting and Print Receipt substates. Once completed, the system transitions back to Idle and becomes ready for the next transaction.

The diagram shows multiple transitions for cancellation, server errors, card validation failures, and success paths, illustrating a complete payment lifecycle from initiation to receipt generation.

Simple Use-Case Diagram:



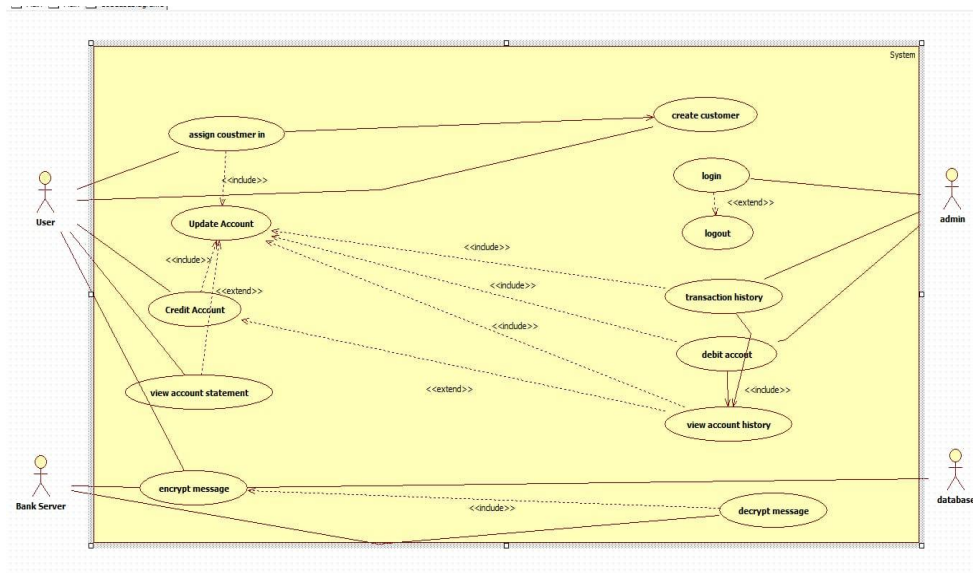
This use case diagram represents a Merchant's Credit Card Processing System interacting with two external actors: the Merchant's Bank and the Customer's Credit Card Bank. The system supports key payment-processing functions such as Authorizing, Capturing, Crediting, Voiding, and Verifying card transactions.

Each use case corresponds to a critical step in the transaction lifecycle.

- Authorizing requests approval from the Customer's Credit Card Bank.
- Capturing secures the approved funds for the merchant.
- Crediting handles refunds back to the customer.
- Voiding cancels a transaction before settlement.
- Verifying checks card validity and available balance.

The system communicates with both banks during these operations to ensure secure, authenticated, and accurate processing of credit card payments.

Advance Use-Case Diagram:



This use case description outlines a secure banking or credit account management system involving four main actors: the User, Admin, Bank Server, and Database. Users can log in, manage their accounts, credit or debit funds, and view statements or transaction history. Admins perform higher-level operations such as creating customer accounts, managing transactions, and performing debit actions. The Bank Server ensures secure communication by encrypting messages, while the Database stores account records and decrypts incoming information. Key use cases include creating customers, updating accounts, performing credit/debit operations, viewing account details, and securely transmitting data, with the system assigning a unique customer ID during account creation.

Scenarios

1: Make Payment

The customer selects the “Make Payment” option. The system displays the total bill amount.

The customer chooses a payment method (UPI, card, banking, etc.). The system sends the payment request to the payment gateway.

The customer enters payment details.

The payment gateway validates the details and processes the transaction. On success, the system

updates the payment status.

A receipt is generated and shown to the customer.

2: Create Customer Account

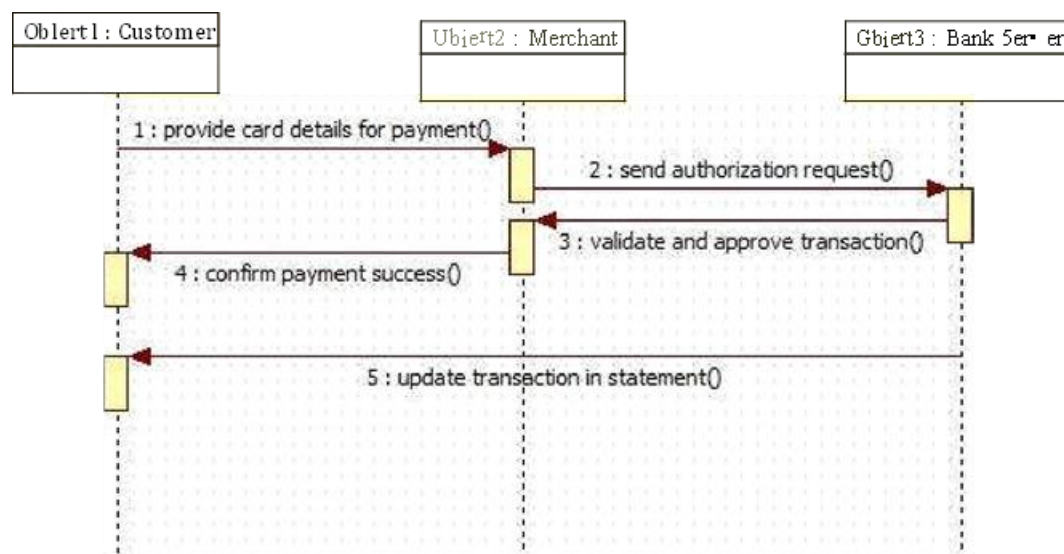
The admin selects the “Create Customer” option. The system displays a form to enter customer details.

The admin enters name, ID proof, contact details, and address. The system validates the information.

A unique customer ID is assigned automatically.

The customer account is created and stored in the database. The system confirms successful registration.

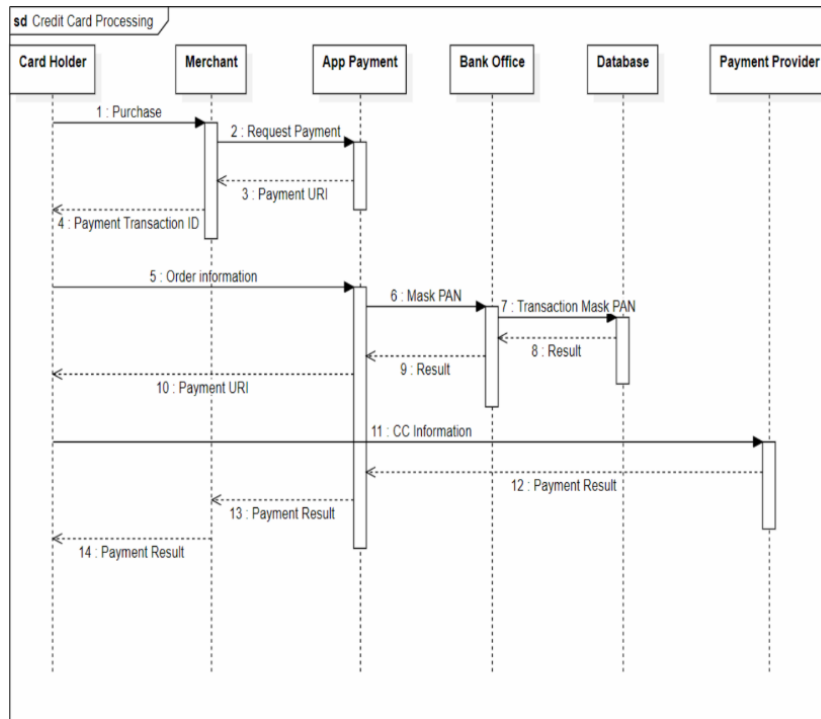
Simple Sequence Diagram:



The simple sequence diagram shows the basic steps involved when a customer makes a payment using a credit card. The process starts when the Customer enters card information, which is then sent to the Credit Card system. The credit card activates the request and sends it to the Payment Gateway. The payment gateway generates the transaction and forwards it to the internal processing object. Once the transaction is approved, a Transaction Successful message is sent back through the payment gateway to the credit card system. The credit card then informs the customer that the

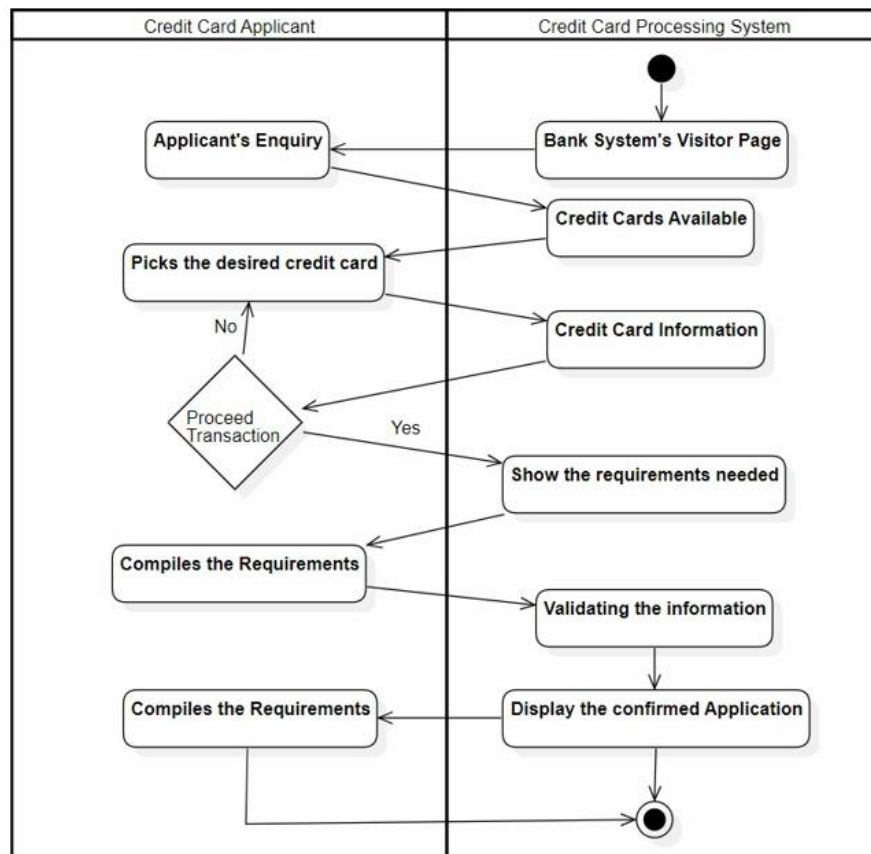
payment is successful, and the transaction is marked as complete. This diagram only focuses on the essential steps: entering card data, verifying payment, and completing the transaction.

Advanced Sequence Diagram



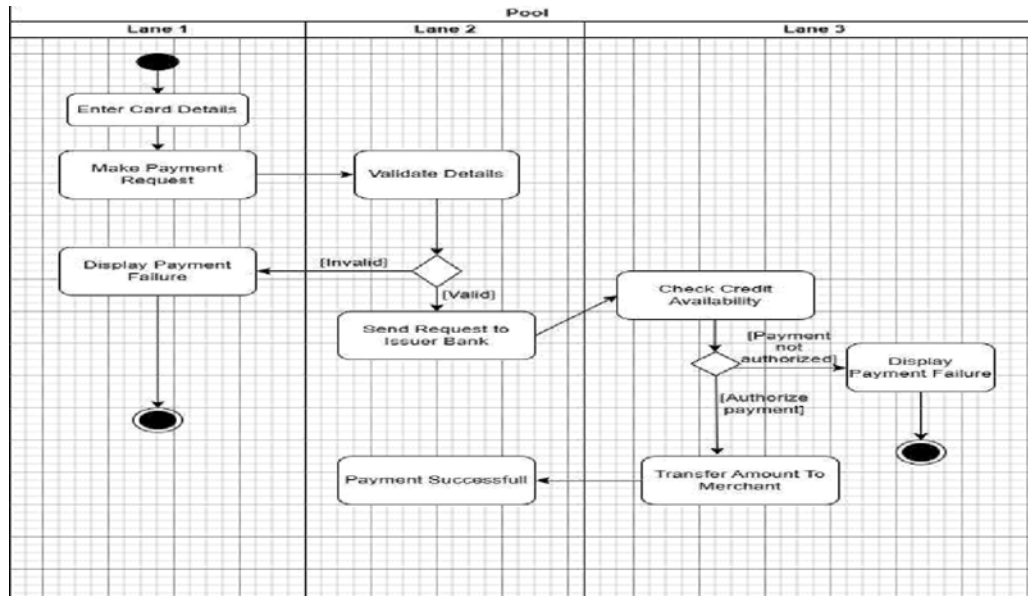
The sequence diagram illustrates the complete flow of credit card processing across multiple components: Card Holder, Merchant, App Payment, Bank Office, Database, and Payment Provider. It shows how a purchase request triggers a chain of interactions—including payment request creation, PAN masking, data validation, and final payment authorization. Each step is represented with messages exchanged between actors, ending with the final payment result being communicated back to the card holder. The diagram emphasizes the security step of masking the PAN and the coordination needed among systems to complete an online payment transaction.

Activity diagram:



The sequence diagram shows how a credit card applicant interacts with the bank's credit card processing system to apply for a card. The applicant begins by making an enquiry, and the system responds by displaying available credit cards. After reviewing the options, the applicant selects a desired card and decides whether to proceed. If proceeding, the system presents the necessary requirements, which the applicant gathers and submits. The system then validates the submitted information and, once confirmed, displays the final application result back to the applicant, completing the process.

Advanced Activity Diagram



The advanced activity diagram presents a more detailed workflow of how a credit card application is handled. It begins when an applicant makes an inquiry about credit cards. The credit card processing system responds by displaying available card options. The applicant then selects a card and decides whether to proceed. If they continue, the system shows the required documents and information needed for application. The applicant gathers and submits these requirements. The system then validates the submitted information, checking eligibility criteria. Once everything is confirmed, the system displays the approved or completed application. This diagram covers more steps and interactions compared to the simple one and offers a clearer view of how credit card applications are processed from inquiry to validation.

3. Library Management System

Problem Statement

In many colleges and institutions, library management is still carried out manually using registers and paper records. This causes problems such as loss of records, difficulty in tracking issued/returned books, and delay in updating student accounts. Students often face inconvenience in finding whether a book is available or not. Therefore, there is a need for a computer-based Library Management System that automates the entire process of book issue, return, and catalog maintenance.

SRS-Software Requirements Specification

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to provide a detailed description of the Library Management System (LMS). It defines the system objectives, requirements, and features in order to help developers and testers in creating a complete and efficient application.

1.2 Scope of this Document

The LMS will automate the functions of a library such as book cataloging, member registration, book issue/return, fine calculation, and report generation. The system will save time for librarians, reduce errors, and make it easier for students to access information. The users of the system will include librarians, students, and administrators.

1.3 Overview

The system will provide a central database to maintain records of all books, members, and transactions. It will also provide search facilities to check book availability. The application will make the library operations faster, more reliable, and more transparent.

2. General Description

The system will be used by:

- Students: To search for books and check their library accounts.

- Librarians: To manage book issue/return and maintain records.
- Administrators: To oversee overall functioning and generate reports.

The objective of the system is to provide a user-friendly solution for managing all library-related tasks in a single platform.

3. Functional Requirements

1. Member Registration and Management
2. Book Catalog Management (add, update, delete books)
3. Book Issue and Return Management
4. Fine Calculation for late returns
5. Search Facility for books by title/author/category
6. Report Generation (books issued, overdue books, fines collected)

4. Interface Requirements

- The system will provide a Graphical User Interface (GUI) for librarians and students.
- Students will be able to log in to check their account status.
- Database connectivity (e.g., MySQL) will be used to store all library records.

5. Performance Requirements

- The system should handle up to 10,000 book records efficiently.
- It should process book issue/return transactions in less than 2 seconds.
- It should support multiple users accessing the system at the same time.

6. Design Constraints

1. Should be developed using Java/Python with a relational database.
2. Must follow basic security rules for student and book records.
3. Should work on college computer labs (Windows/Linux systems).

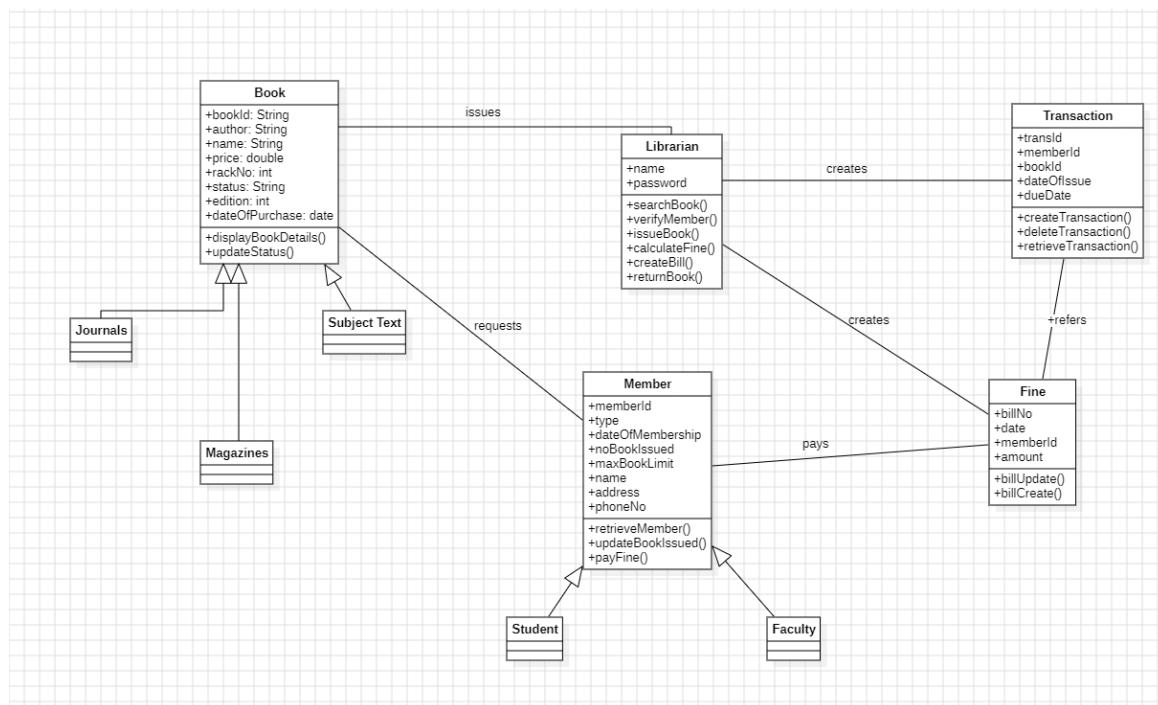
7. Non-Functional Attributes

1. Security: Only registered users can access the system.
2. Reliability: Records should not be lost or corrupted.
3. Scalability: Should handle increasing number of books and students.
4. Usability: Easy-to-use interface for students and librarians.
5. Portability: Should run on multiple operating systems.

8. Preliminary Schedule and Budget

The development of the LMS is expected to take 3 months with a budget of \$15,000. This includes requirement gathering, design, development, testing, and deployment.

Class Diagram

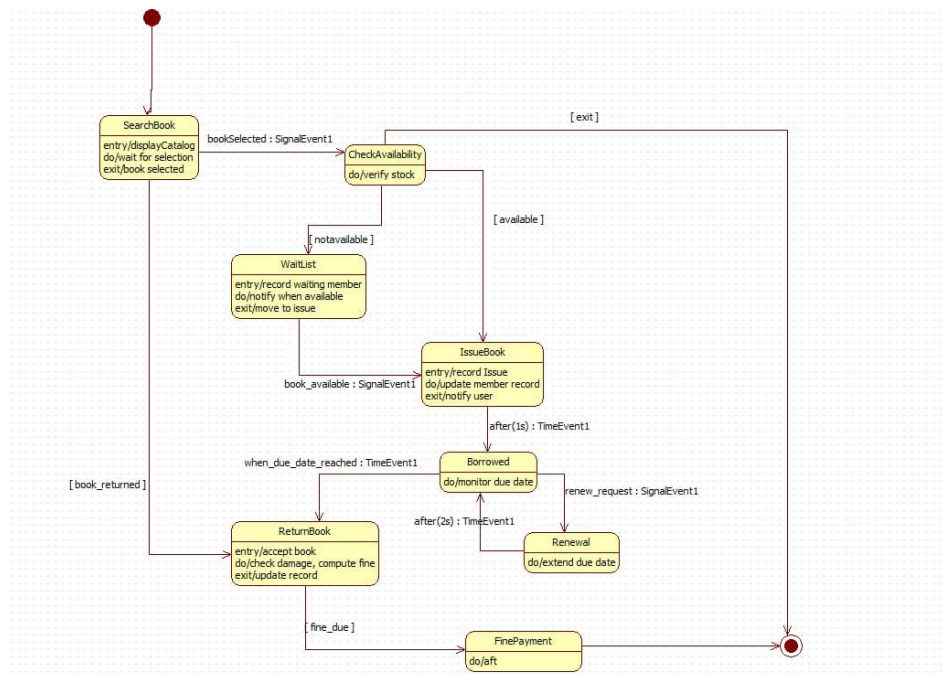


The class diagram represents a Library Management System where books, members, librarians, and transactions interact to manage library operations. The Book class stores details such as author, edition, price, and purchase date, and it is specialized into different types like Journals, Magazines, and Subject Text through inheritance. Members of the library—categorized as

Students and Faculty—send book requests and borrow books. The Librarian plays a central role by searching books, verifying members, issuing books, calculating fines, and handling book returns. The librarian also updates the status of books and ensures that library policies like issuing limits are followed.

The system also keeps track of borrowing activities using the Transaction class, which stores the transaction ID, book ID, issue date, and due date. When members return books late, the Fine class is used to generate bills, update fines, and store payment information. Members pay these fines, and the librarian or system records them for future reference. Overall, the class diagram shows how different components—books, members, librarians, transactions, and fines—work together to ensure smooth operation of the library.

Simple State Diagram:



This state machine diagram models the lifecycle of a library book transaction using states, transitions, guard conditions, signal events, and time events.

The process begins in **SearchBook**, where the user browses the catalog and selects a book. A **SignalEvent** triggers the transition to **CheckAvailability**, which verifies stock.

- If the book is not available, the system moves to the **WaitList** state, recording the member and notifying them when the book becomes available.

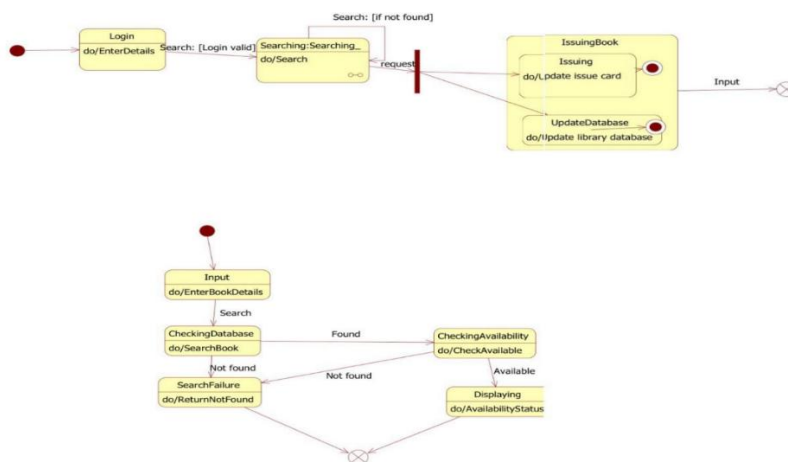
- If the book is available, or becomes available later, the system transitions to IssueBook, updating member records and notifying the user.

After issuing, the book enters the Borrowed state, where the system monitors due dates through TimeEvent triggers.

- A renew_request signal transitions to Renewal, extending the due date and returning to Borrowed.
- When the due date is reached, the flow moves to ReturnBook, where the system accepts the book, checks for damage, and computes fines if applicable.

If a fine is due, the system transitions to FinePayment before reaching the final pseudostate, completing the workflow.

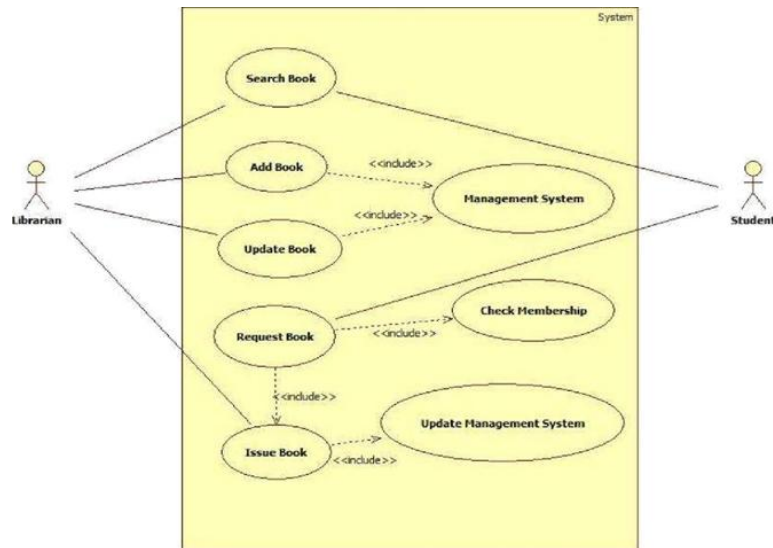
Advanced State Diagram:



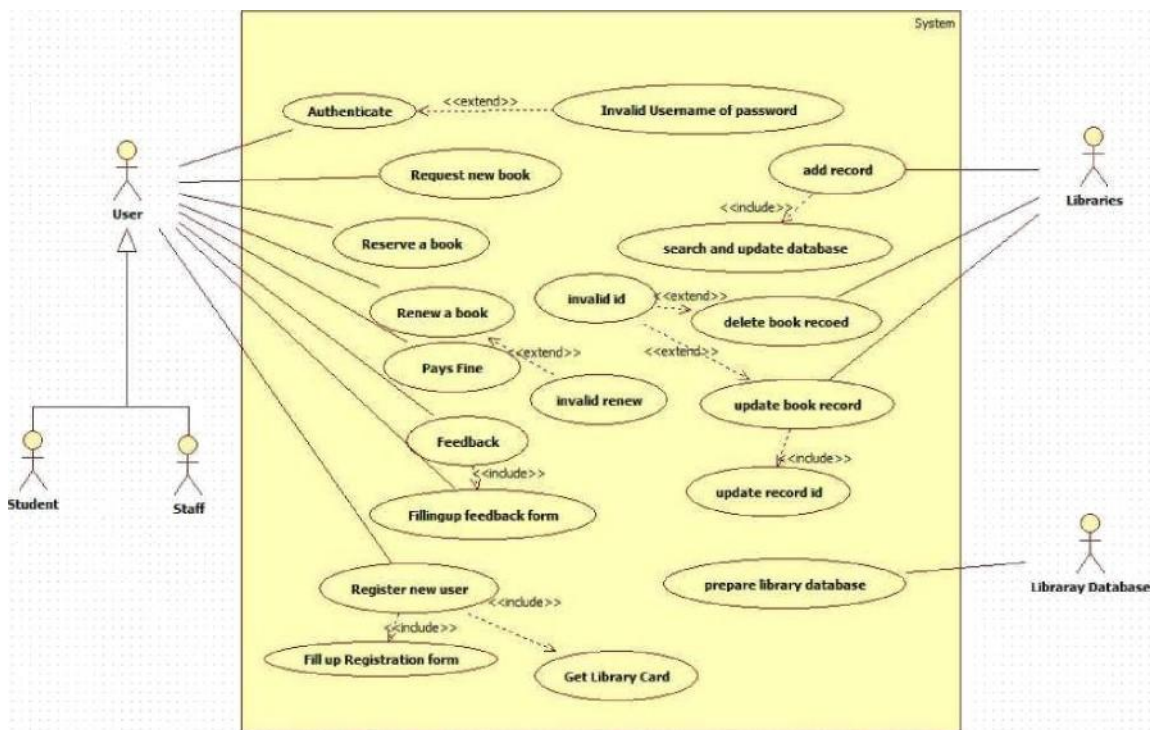
This state machine diagram models the book search and issuing process in a library system. After the user logs in, the system enters the Searching state to locate the requested book. A transition then leads into the IssuingBook composite state, which contains the sequential substates Issuing (updating the issue card) and UpdateDatabase (updating the library records). The lower flow shows the book lookup logic: starting from Input, the system checks the database. If the book is not found, it transitions to SearchFailure and ends. If found, CheckingAvailability verifies stock, and Displaying shows the availability status.

The diagram uses guard conditions, composite states, and decision nodes to represent search, issue, and database-update behaviors.

Use-Case Diagram



Advanced Use Case Diagram



Simple Use Case Diagram

The simple use case diagram focuses on the core interactions in the Library Management System

between the Librarian and the Student. The librarian performs essential tasks such as Searching Books, Adding Books, Updating Book Details, Requesting Books, and Issuing Books. Many of these actions include interactions with the Management System, such as checking membership and updating records. The student mainly interacts through the system to request or search for books. The diagram highlights only the fundamental operations that support the basic flow of library activities—finding, adding, updating, and issuing books—making it easy to understand the primary responsibilities of the librarian and how students request books.

Advanced Use Case Diagram

The advanced use case diagram presents a more detailed and comprehensive view of the library system by involving multiple actors—User (Student/Staff), Libraries, and the Library Database. It includes a wider range of use cases such as Authentication, Requesting New Books, Reserving Books, Renewing Books, Paying Fines, Providing Feedback, Registering New Users, and Filling Forms. The system also manages book records using operations like Adding, Updating, and Deleting Records, as well as preparing the library database. Several use cases use *include* and *extend* relationships to show dependencies and optional behaviors such as invalid login, invalid ID, or invalid renewal. This advanced diagram gives a full picture of how users interact with the system, how library data is maintained, and how operations extend beyond simple issuing and returning of books.

Scenarios

1: Issue Book

The librarian selects the “Issue Book” option.

The librarian enters the member ID and book ID.

The system checks the member's status and borrowing limit.

The system verifies the availability of the book.

If valid, the system updates the issue record and marks the book as borrowed.

The librarian hands the book to the member.

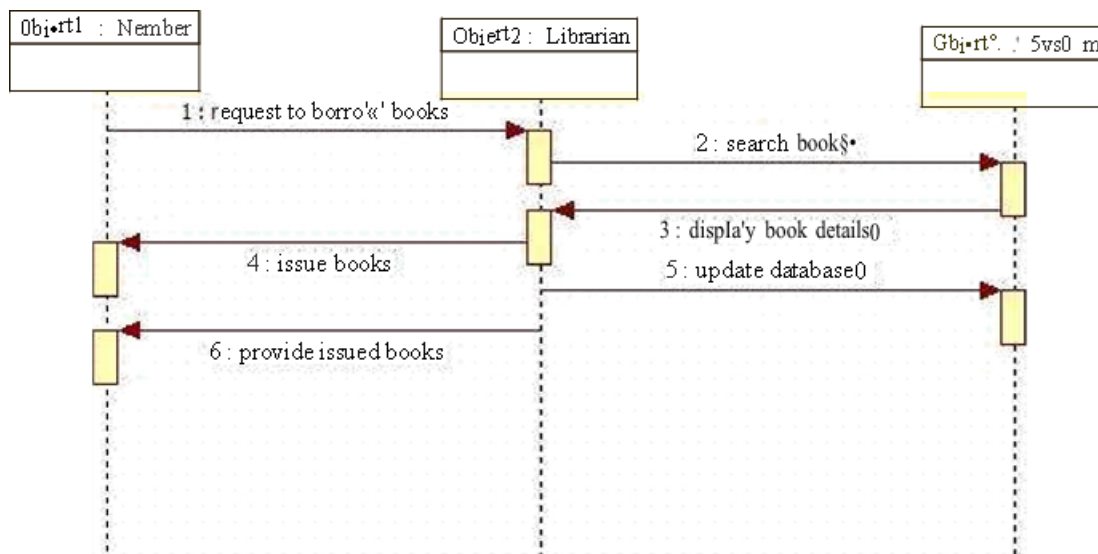
2: Search Book

The student selects the “Search Book” option. The system prompts for the book title/author/ID. The student enters the details.

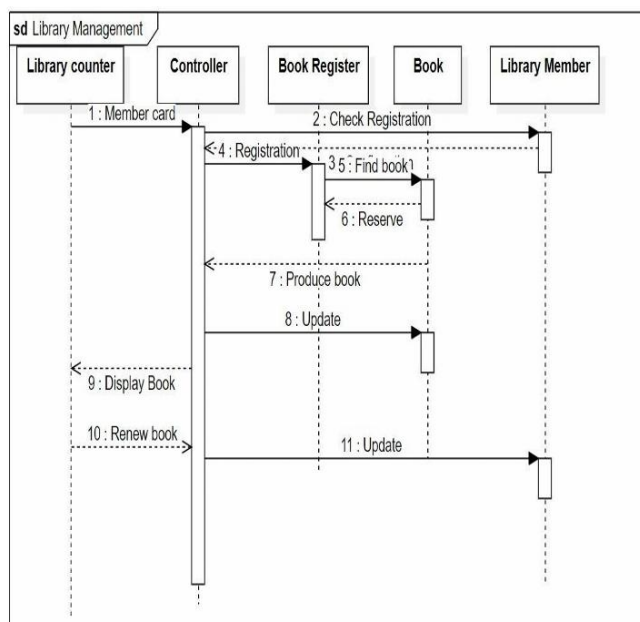
The system scans the database for matching books.

The system displays the availability status (Available / Issued / Not Found).

Simple sequence diagram



Advanced Sequence Diagram:



Simple Sequence Diagram (Member — Librarian — System)

The simple sequence diagram shows the basic process of borrowing a book in the library. The interaction begins when the Member requests to borrow a book. The Librarian receives this request and sends a command to the System to search for the book. The system returns the book details, after which the librarian proceeds to issue the book to the member. Once the book is issued, the librarian updates the library database, and finally the member receives the issued book. This diagram focuses only on the essential steps—requesting, searching, issuing, and updating—making it easy to understand how a book is borrowed in a simple workflow.

Advanced sequence Diagram

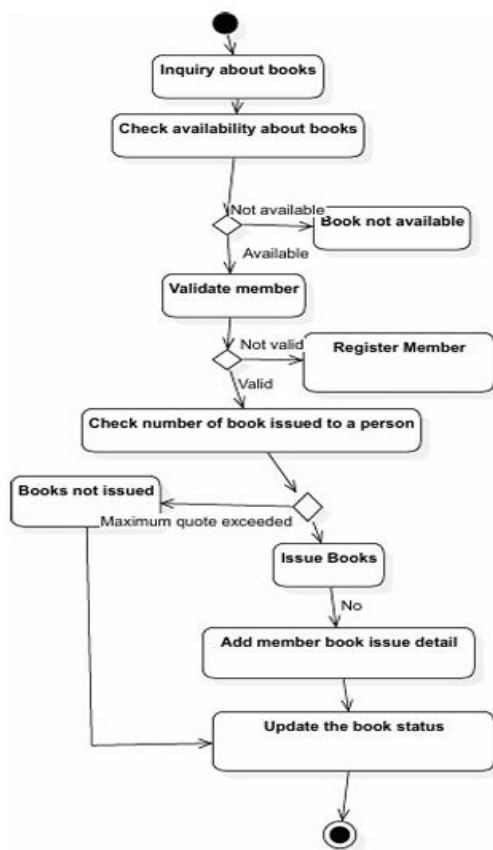
This sequence diagram illustrates the interaction between the Library Counter, Controller, Book Register, Book, and Library Member during the book issuing and renewal process.

The workflow begins when the member presents their Member Card. The controller verifies membership by sending a Check Registration request to the Book Register. After confirming registration, the system searches for the book, reserves it, and the Book Register instructs the Book entity to update its status.

The controller then produces the book for the member and displays the book details. When the user requests renewal, an update message is sent again to the Book entity to extend due dates.

Overall, the diagram shows synchronous messages, database updates, and control flow across components for registration validation, book reservation, issuing, and renewal operations.

Simple Activity diagram



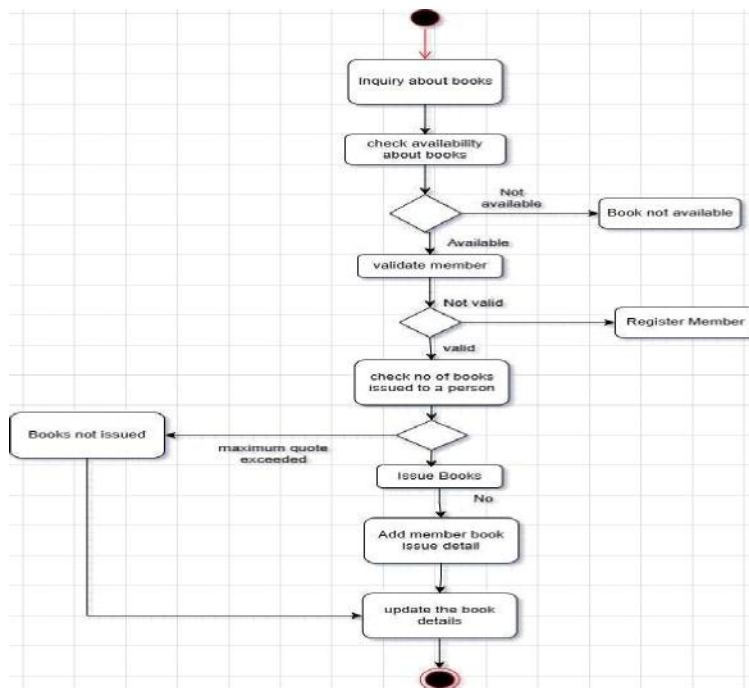
This activity diagram models the book issuing workflow in a library system. The process starts with an inquiry and proceeds to check book availability. A decision node handles the “available/not available” condition. If available, the system moves to validate the member, with another decision directing invalid users to register member.

Once validated, the system checks the number of books already issued to enforce the borrowing quota. If the quota is exceeded, the flow moves to Books not issued; otherwise, it proceeds to Issue Books.

After issuing, the system updates the member’s book-issue details and then performs Update the book status, leading to the final node.

The diagram uses decision nodes, control flows, and activity transitions to represent conditional paths and sequential operations in the book issuing process.

Advanced Activity Diagram



The advanced activity diagram provides a more detailed and complete view of the book-issuing workflow. In addition to checking availability and validating the member, it clearly shows decision points such as book unavailability, invalid member status, and quota limits. It also includes additional actions like adding member book issue details, updating the complete book record, and handling both “Book not issued” and “Issue Books” outcomes. By showing all alternative paths and internal updates, the advanced diagram presents the full operational logic of how the library processes book inquiries, member eligibility, quota checks, issuing actions, and record maintenance. It reflects the complete backend process of issuing a book in a real library system.

4. Stock Maintenance System

Problem Statement

In many shops, warehouses, and organizations, stock management is still handled manually using registers or spreadsheets. This often leads to errors such as incorrect stock counts, overstocking, or shortage of items. It also becomes difficult to generate reports and track sales or purchases efficiently. To solve these problems, there is a need for a computerized Stock Maintenance System that automates inventory tracking, stock updates, and reporting.

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to define the requirements for the Stock Maintenance System (SMS). It provides details about the system's objectives, features, and functionalities so that developers and testers can design and implement the application effectively.

1.2 Scope of this Document

The Stock Maintenance System will help organizations keep accurate records of items available, purchased, or sold. It will allow managers to track stock levels, generate reports, and predict shortages in advance. The system will reduce human error, save time, and increase efficiency in managing inventory.

1.3 Overview

The SMS will maintain a central database of stock items, purchases, and sales. It will include modules for updating stock, managing suppliers/customers, and generating reports. The system will support real-time stock tracking and ensure that the data is always accurate and up to date.

2. General Description

The system will be used by:

- **Warehouse/Store Managers:** To update stock and track inventory.
- **Employees:** To enter sales and purchase records.

- **Administrators:** To generate detailed stock and sales reports.

The system's objective is to provide a fast and reliable way to maintain stock information and reduce manual paperwork.

3. Functional Requirements

1. Item Registration (add/update/delete stock items)
2. Purchase Entry and Update in stock
3. Sales Entry and Stock Reduction
4. Alert System for Low Stock Levels
5. Supplier and Customer Management
6. Report Generation (stock status, sales, purchases, profit/loss)

4. Interface Requirements

- The system will have a **GUI** for users to add/update/view stock.
- The backend database (e.g., MySQL) will maintain stock records.
- Managers will have access to generate reports, while employees will have limited access.

5. Performance Requirements

- The system should support up to **20,000 stock items**.
- It should process updates within **2 seconds**.
- It should allow multiple employees to work simultaneously without conflicts.

6. Design Constraints

1. Should be developed using Java/Python with a database like MySQL.
2. Must follow basic security measures to protect inventory records.
3. Should run on standard desktop computers with Windows/Linux OS.

7. Non-Functional Attributes

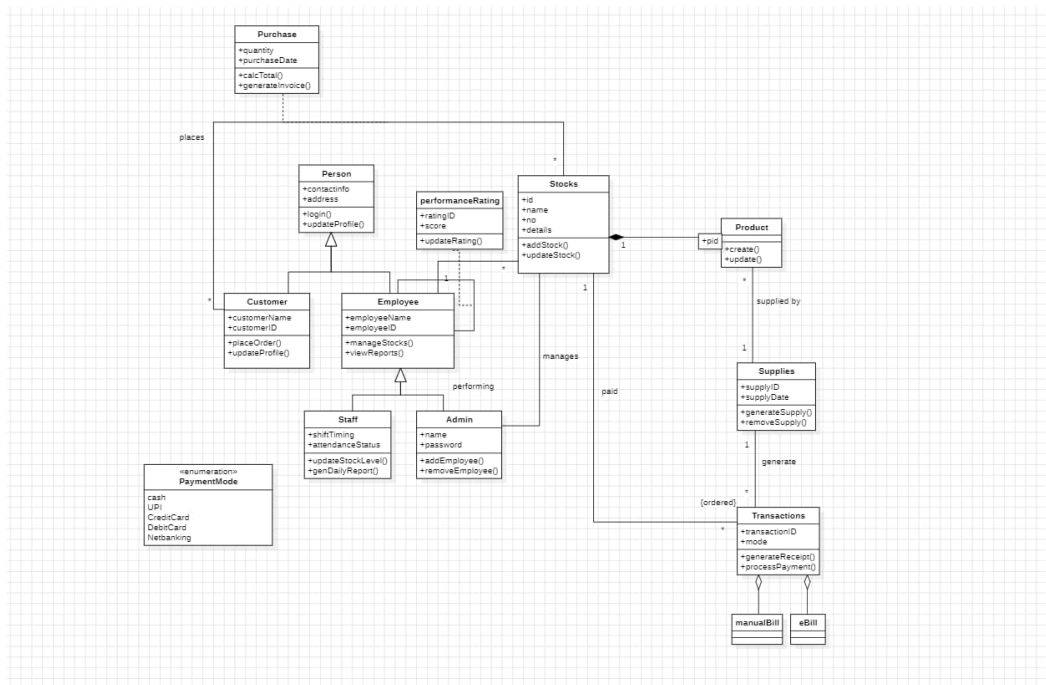
1. **Security:** Only authorized users can modify stock.
2. **Reliability:** System should not lose data even during crashes.

3. **Scalability:** Must support future expansion of items and users.
4. **Usability:** Easy to use for employees with minimal training.
5. **Portability:** Compatible with different operating systems.

8. Preliminary Schedule and Budget

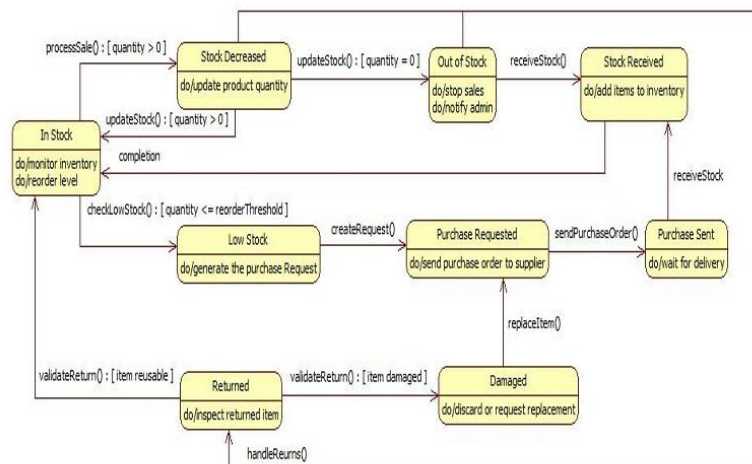
The Stock Maintenance System is expected to take **4 months** to develop with an estimated budget of **\$18,000**. This includes requirement analysis, design, coding, testing, and final deployment.

Class Diagram:



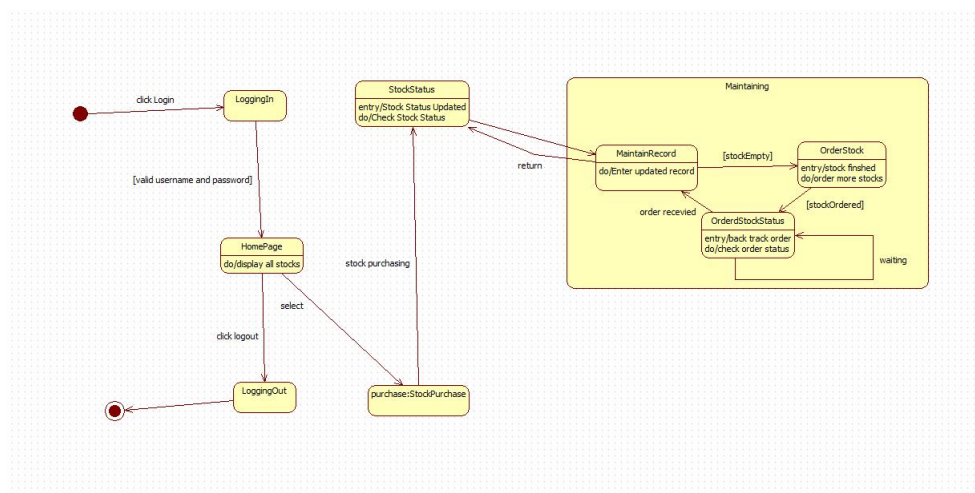
This class diagram represents a retail management system where customers place purchase orders for products, employees manage stock levels, and admins oversee staff and system operations. The **Person** class generalizes customers and employees, while employees may have performance ratings and belong to roles such as **Staff** or **Admin**. **Products** are supplied through **Supplies** and tracked in **Stocks**, which employees update and manage. **Transactions** are generated after purchases, using various payment modes, and invoices or receipts are created accordingly. Overall, the diagram shows how purchasing, stock management, employee functions, and supply operations interact within one integrated system.

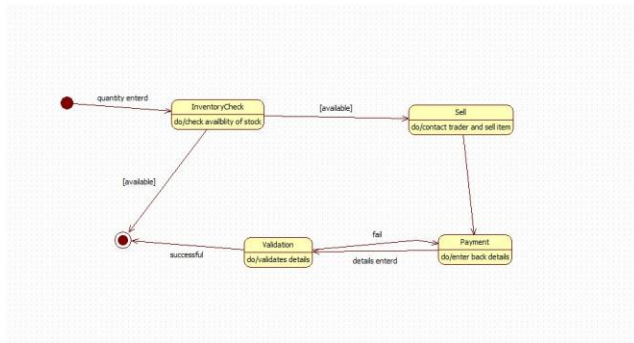
Simple State Diagram:



This state diagram illustrates the various stages an item goes through in an inventory or order management system, from being available to the possibility of being out of stock. The process begins when an item is requested, transitioning the item from "Available" to "Reserved" if the quantity is sufficient, or returning to "Available" if a reservation is canceled. After payment, the item is "Dispatched" and moves to "Delivered" once confirmed by the customer. If returned within the allowed period, it enters the "Returned" state for inspection and potential restocking. At any point, if stock runs out, the item moves to the "Out of Stock" state, where reordering and restocking actions are managed.

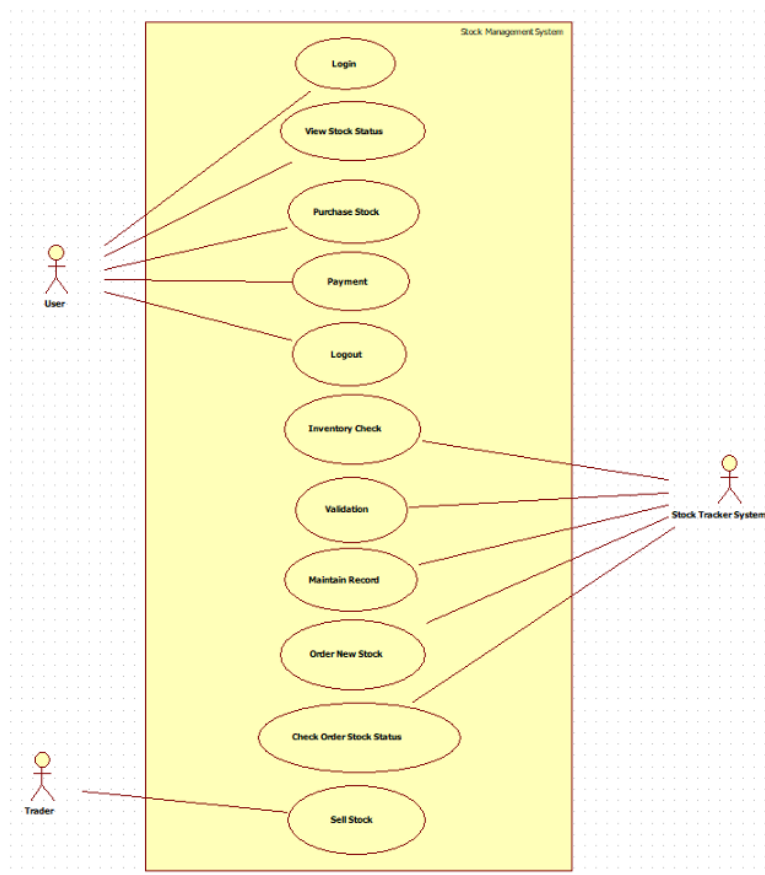
Advanced State Diagram





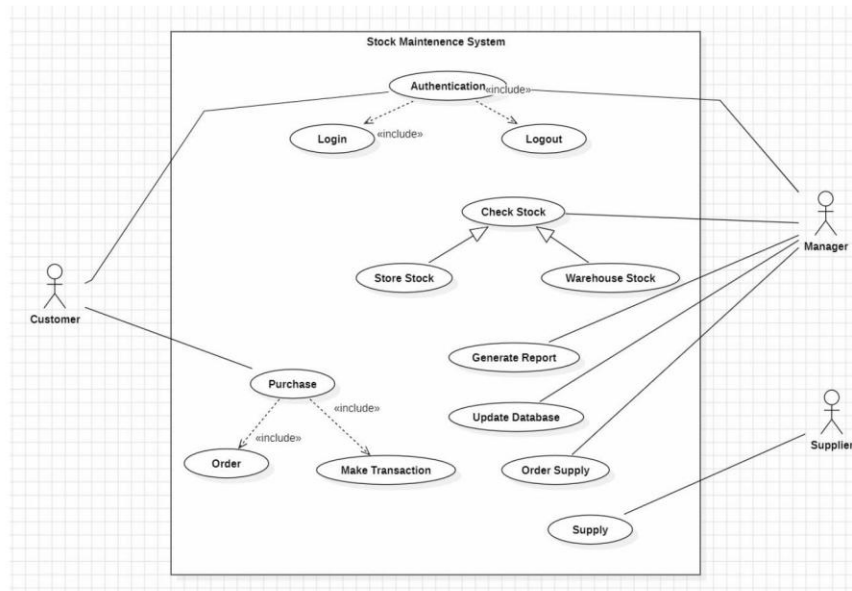
This state diagram illustrates the flow of a stock management system, beginning with a user logging in and accessing the homepage where stock information is displayed. From there, the user can view stock status or proceed to purchase stock. The system then transitions into the maintenance process, where stock records are updated, low stock triggers new stock orders, and order status is tracked until completion. Once updates are made and the system returns to the main flow, the user may continue managing stock or log out, completing the session.

Simple use case diagram



This use case diagram illustrates the interactions between two primary actors—Store Manager and Supplier—in a stock management system. The Store Manager handles inventory operations such as adding, updating, and viewing stock levels, as well as generating reports for analysis. The Supplier is responsible for requesting supply and delivering stock to the store. Each actor is linked to specific system functions, clarifying their roles and responsibilities. This visual representation helps in understanding system requirements, streamlining workflows, and guiding software development. It ensures that all user interactions are captured, making the system more efficient, user-centric, and aligned with business objectives.

Advanced Use-Case Diagram:



This use case diagram represents a Stock Maintenance System involving three main actors: the Customer, Manager, and Supplier. Customers can log in, check stock, make purchases, place orders, and complete transactions. Managers oversee key system functions such as checking store and warehouse stock, generating reports, updating the database, and ordering supplies when needed. Suppliers provide the requested stock through the supply process. Authentication is included in the login and logout processes, while purchasing includes order placement and transaction handling. Overall, the diagram shows how different users interact with the system to manage stock, fulfill orders, and maintain smooth operations.

Scenario for Product Ordering System

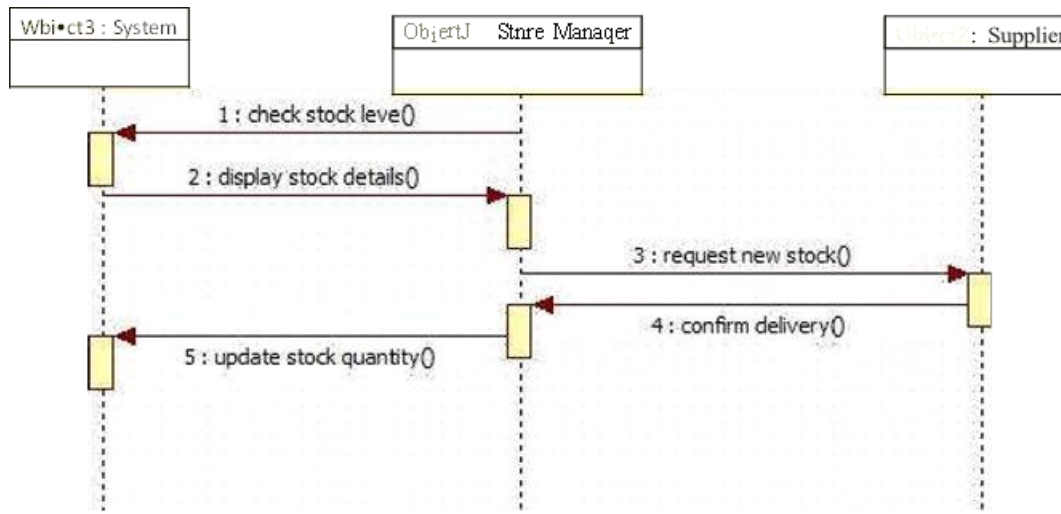
Scenario 1: Successful Purchase and Stock Update (Happy Path)

- Purchase department raises a purchase requisition for 500 units of Item-X
- Supplier accepts PO and delivers 500 units with proper invoice
- Warehouse staff receives goods, performs quality check —Passed
- Staff scans/enters GRN (Goods Receipt Note) into the system
- System automatically increases stock level of Item-X by 500
- Stock value updated in inventory ledger
- Finance receives GRN copy and processes supplier payment
- Re-order level alert for Item-X automatically turns off
- Dashboard shows updated stock: Item-X = 1,200 units

Scenario 2: Stock Rejection and Return (Exception Scenario)

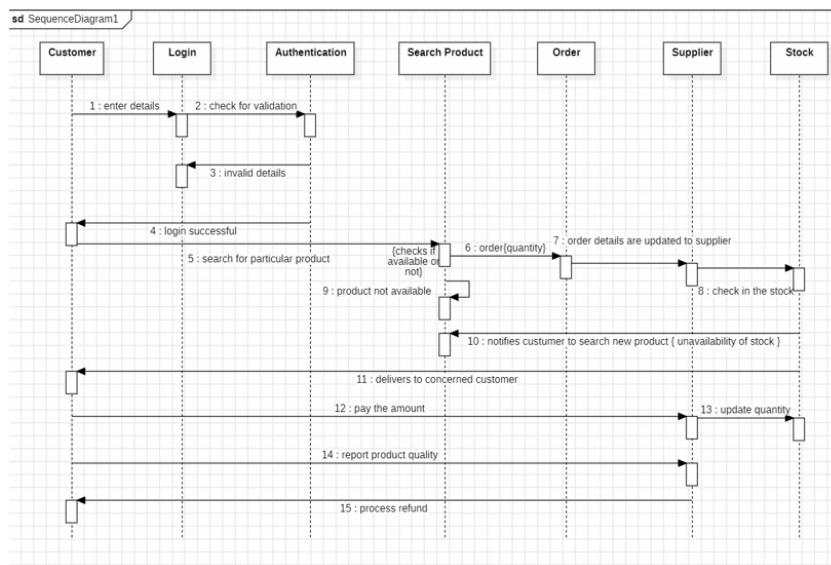
- Supplier delivers 300 units of Item-Y, but 80 units found damaged
- Warehouse staff performs quality check —8m units United
- Staff creates Partial GRN for 220 units only
- System adds only 220 units to stock, blocks 80 units
- Return Material Authorization (RMA) generated automatically
- Rejected 80 units returned to supplier with Debit Note
- Stock level of Item-Y updated correctly (only +220)
- Supplier acknowledges return, issues Credit Note
- Inventory report reflects accurate stock and pending returns
- Low-stock alert remains active since actual receipt is below requirement

Simple Sequence Diagram



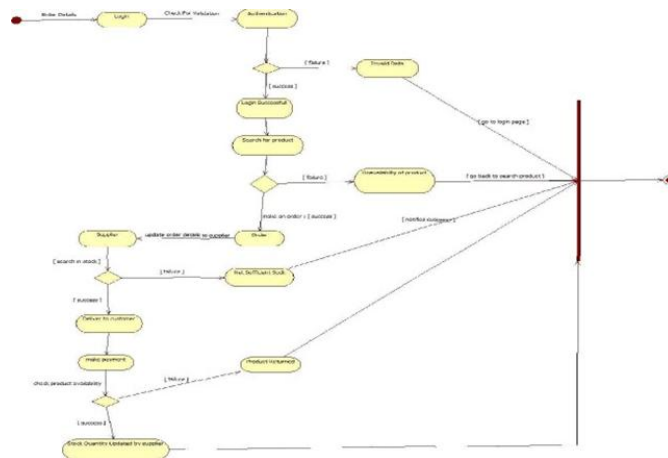
This UML sequence diagram illustrates the interaction between the Store Manager, Supplier, and System in a stock management workflow. The Store Manager initiates the process by checking stock levels through the System, which responds by displaying stock details. If stock is insufficient, the Manager requests new stock from the Supplier, who confirms delivery. Following this, the Manager updates the stock quantity in the System to reflect the new inventory. This sequence clearly outlines the communication flow and operational dependencies among the actors and system components, helping developers understand system behavior and ensuring accurate implementation of inventory-related functionalities.

Advanced Sequence Diagram

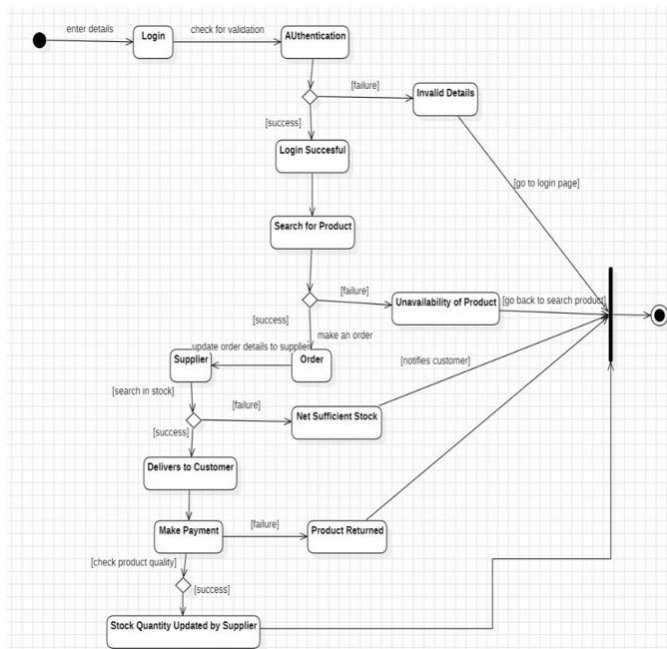


This sequence diagram illustrates the complete flow of a customer's interaction with the product ordering system—from login and product search to ordering, delivery, payment, and potential refund. It shows how different system components, including authentication, product search, order processing, supplier, and stock management, coordinate to validate user details, check product availability, fulfill orders, update stock, and handle after-sales services such as quality reporting and refunds.

Simple Activity Diagram



Advanced Activity Diagram:



This activity diagram illustrates the workflow of a product ordering system, beginning with the customer logging in and having their details authenticated. After a successful login, the customer searches for a product, and the system checks its availability. If the product is available, an order is placed and sent to the supplier, who verifies stock levels. When stock is sufficient, the product is delivered to the customer, who then makes the payment. If product quality issues arise, returns can be processed and the supplier updates stock quantities accordingly. In cases of invalid login details, unavailable products, or insufficient stock, the diagram shows alternate paths that guide the user back to appropriate steps, ensuring a complete and well-managed process flow.

5.Passport Automation System

Problem Statement

The traditional passport application process is often slow and inefficient due to manual handling of applications, long queues, and paperwork. Applicants face problems such as delays in approval, loss of documents, and lack of transparency in the process. Similarly, authorities struggle to manage large volumes of applications effectively. To overcome these issues, there is a need for a **Passport Automation System** that digitizes the entire process, making it faster, secure, and more transparent.

1. Introduction

1.1 Purpose of this Document

The purpose of this document is to describe the requirements of the Passport Automation System (PAS). It defines the objectives, features, and system functionalities that will help developers and testers build the application in an efficient and reliable manner.

1.2 Scope of this Document

The Passport Automation System will allow applicants to submit applications online, upload required documents, track the status of their application, and receive notifications. For authorities, the system will provide tools for verifying documents, processing applications, and issuing passports. The system will reduce paperwork, save time, and provide better services to citizens.

1.3 Overview

The PAS will consist of modules for application submission, verification, payment processing, and status tracking. The system will ensure security of applicant data, reduce manual effort, and provide transparency throughout the process.

2. General Description

The system will be used by:

- **Applicants:** To apply for a passport, upload documents, and check application status.

- **Passport Officers:** To verify documents and approve/reject applications.
- **Administrators:** To oversee the entire system and generate reports.

The main objective of the system is to digitize and simplify the passport issuance process.

3. Functional Requirements

1. Online Application Submission with document upload
2. Payment Gateway for application fees
3. Verification Module for officers to check documents and details
4. Status Tracking and Notifications for applicants
5. Appointment Scheduling for interviews/verification
6. Report Generation for administrators

4. Interface Requirements

- A **web-based interface** for applicants to apply online.
- A secure portal for officers to verify and process applications.
- Database connectivity (e.g., MySQL/Oracle) to store applicant records and documents.

5. Performance Requirements

- The system should handle **thousands of applications per day**.
- The response time for each request should be **under 2 seconds**.
- The system should be available **24/7** for applicants.

6. Design Constraints

1. Must follow government rules and policies for passport applications.
2. Must ensure data security using encryption techniques.
3. Should support integration with government ID databases.

7. Non-Functional Attributes

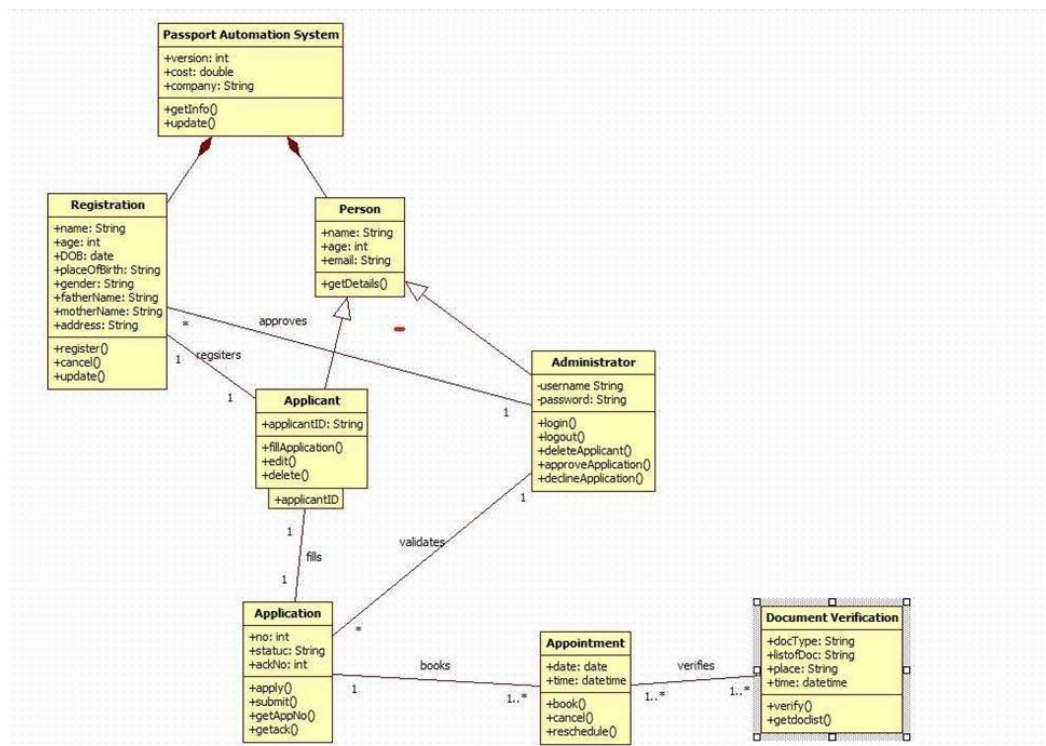
1. **Security:** Sensitive applicant data must be encrypted.
2. **Reliability:** The system should have 99.9% uptime.

3. **Scalability:** Must support future increases in number of applicants.
4. **Usability:** Easy to use for applicants with minimal computer knowledge.
5. **Portability:** Compatible with web browsers on multiple platforms.

8. Preliminary Schedule and Budget

The Passport Automation System is expected to take **6 months** for development with an estimated budget of **\$60,000**. This covers requirements gathering, design, coding, testing, security validation, and deployment.

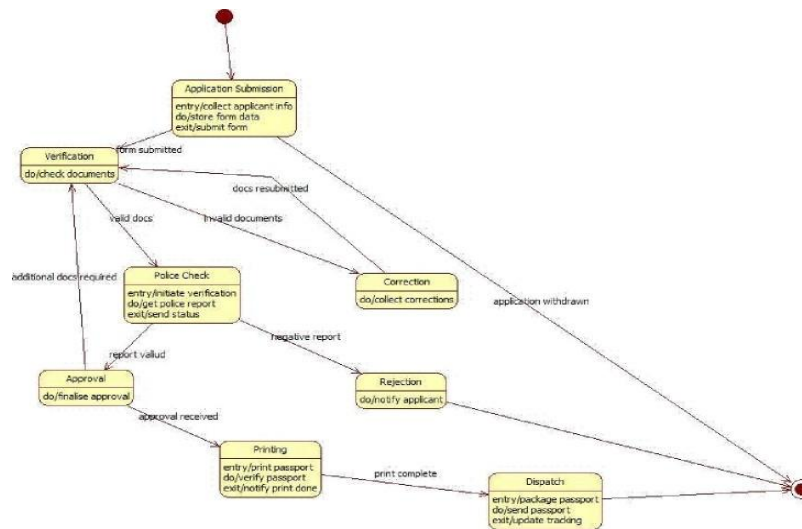
Class Diagram:



This class diagram represents an online Passport Automation System. An Applicant (inheriting basic details from Person and Registration) fills and submits an Application, which an Administrator validates, approves, or declines. Once approved, the applicant books an Appointment for document verification. At the center, Document Verification staff verify the applicant's original documents (docType, place, etc.) during the scheduled slot. The system maintains applicant info, tracks application status, manages appointments, and logs all actions

(register, update, cancel, reschedule) while ensuring only verified and approved applicants proceed to the final appointment stage.

Simple State Diagram

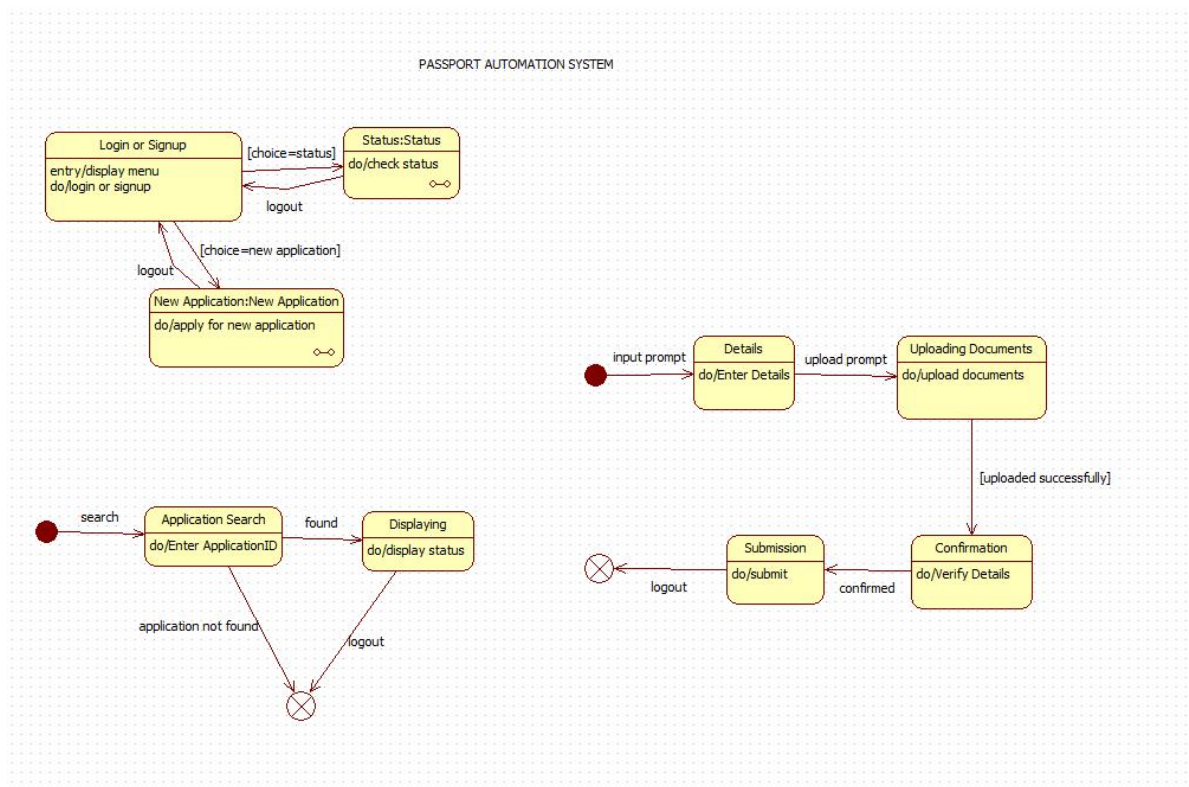


This state machine diagram represents the end-to-end workflow of a Passport Application Process. The process starts with Application Submission, where the applicant's information and form data are collected. The system then transitions to Verification, where submitted documents are checked. If documents are invalid or incomplete, the flow moves to Correction, allowing the applicant to resubmit corrected documents. If valid, the system proceeds to Police Check, where police verification is initiated and the status report is received.

Based on the police report, the application either moves to Approval (positive report) or Rejection (negative report). After approval, the process transitions to Printing, where the passport is printed and verified. Once printing is complete, the final state Dispatch packages and sends the passport, updating tracking details before reaching the end state.

The diagram captures conditional flows, document resubmission, verification loops, and final dispatch operations within the passport processing cycle.

Advanced State Diagram



This state machine diagram models the workflow of a Passport Automation System, starting from user login and moving through application submission and status checking.

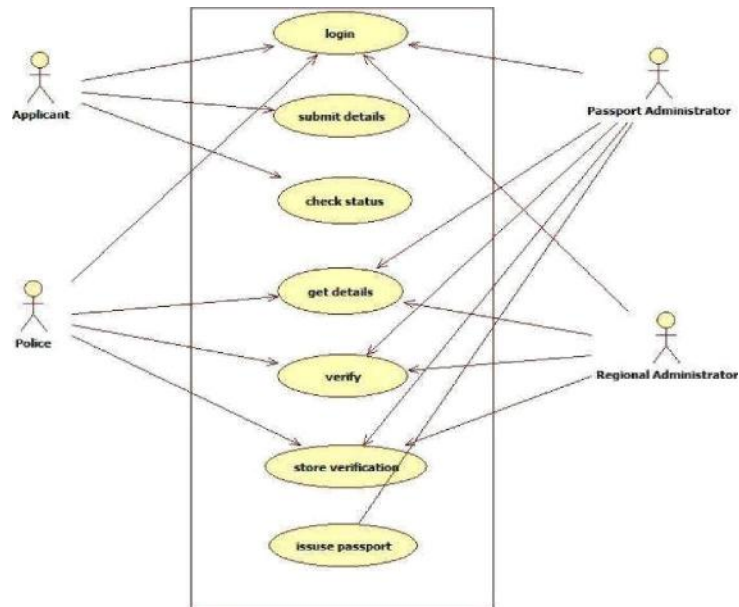
Users begin in the Login or Signup state, where they choose to log in, sign up, or start a new application. If they opt to check progress, the system transitions to the Status state to retrieve application status. If they choose to apply, the system enters the New Application state.

For new applications, the process continues with Details, where the applicant enters personal information, followed by Uploading Documents, where required documents are uploaded. Once successfully uploaded, the flow transitions to Confirmation, where the system verifies the submitted details. After confirmation, the user proceeds to Submission, which completes the application process.

The lower section models Application Search, allowing users to enter an Application ID. If found, the system displays status through the Displaying state; if not found, the flow terminates at a final state.

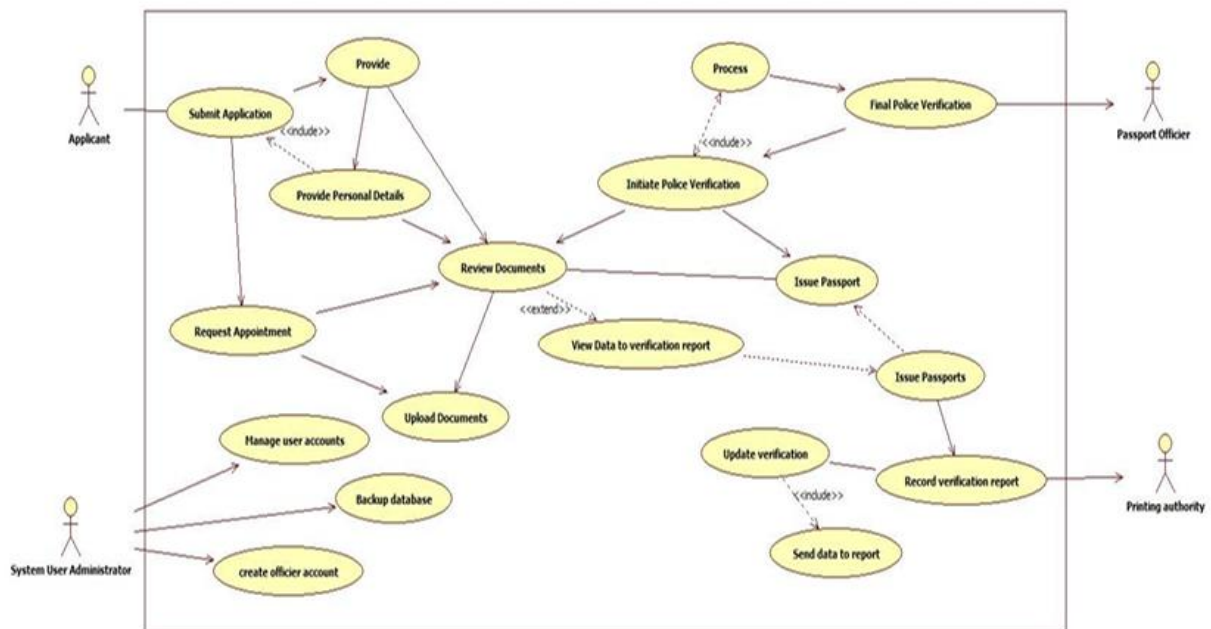
The diagram uses decision points, transitions, and parallel state sections to represent login, application creation, document upload, validation, and status tracking in the passport system

Simple Use case Diagram



This use case diagram represents a Passport Automation System involving three actors: Applicant, Passport Officer, and Admin. The Applicant can Apply for Passport, Schedule Appointment for document verification, and Check Status of their application anytime. The Passport Officer is responsible for Verify Documents during the appointment and subsequently Issue or Reject the Passport based on authenticity and police verification reports. The Admin handles administrative tasks by managing user accounts (Manage Users) and updating application statuses (Update Application Status) throughout the process. The diagram clearly separates citizen-facing, officer-level, and administrative functions, ensuring smooth, role-based interaction with the system.

Advanced Use Case Diagram



This use case diagram represents a Passport Automation System involving three major actors: Applicant, Passport Officer, and System User/Administrator, along with the Printing Authority. The Applicant performs primary tasks such as Submit Application, Provide Personal Details, Upload Documents, and Request Appointment. These actions collectively support the main use case Review Documents, which can extend to providing additional information when needed.

The Passport Officer handles verification-related activities, including Initiate Police Verification, Process verification data, and conduct the Final Police Verification. Based on verification results, the officer proceeds to Issue Passport, while the Printing Authority completes tasks like Record Verification Report, Issue Passports, and Send Data to Report for final printing.

The System User/Administrator oversees administrative functions including Manage User Accounts, Create Officer Account, and Backup Database, ensuring smooth operational support. The diagram clearly differentiates applicant tasks, officer-level verification processes, and administrative responsibilities, ensuring a structured and role-based workflow for passport processing.

Scenarios

Scenario 1: Main Success Scenario (Happy Path)

Applicant registers and logs into the passport portal

Fills application form correctly and uploads clear documents

Pays fees online successfully

Books and attends appointment at PSK on scheduled date

Officer verifies original documents and captures biometrics smoothly

Police verification completed with clear/no-adverse report

Regional authority approves the application

Passport printed and dispatched via Speed Post

Applicant tracks status online and receives passport within 25-30 days

Scenario 2: Alternative Scenario (Rejection due to Adverse Police Report)

Applicant completes registration, form filling and fee payment

Attends appointment; document & biometric verification successful

Police station visits applicant's address for enquiry

System detects pending criminal case / FIR

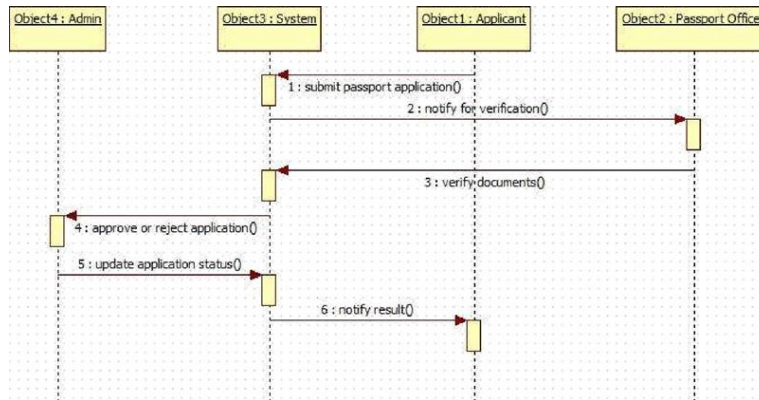
Police submit adverse verification report

Regional authority reviews and rejects the application

Applicant receives SMS + email notification sent with rejection reason

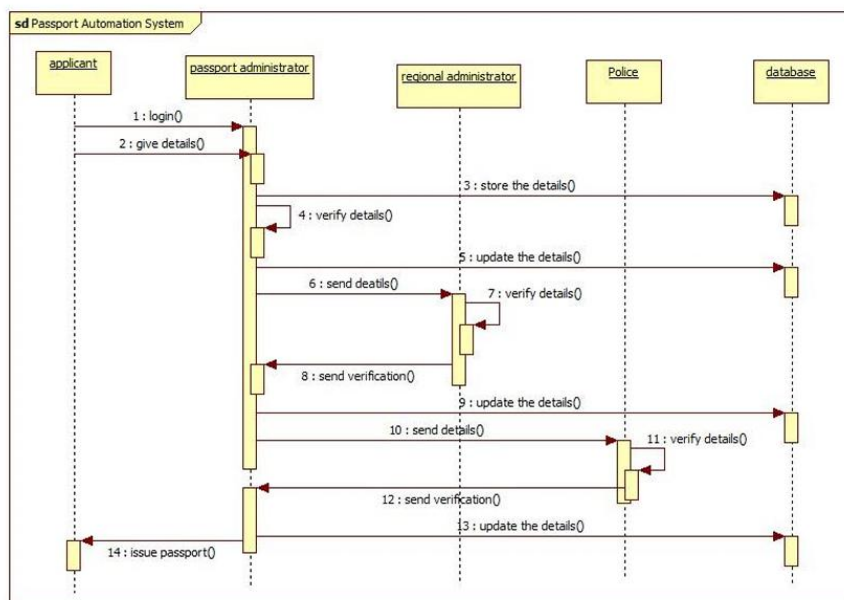
Status updated as “Rejected” on portal

Simple sequence Diagram



This sequence diagram illustrates the interaction flow in a Passport Automation System. The Applicant (Object1) initiates by submitting the passport application to the System (Object3). The System immediately notifies the Passport Officer (Object2) for verification. The Officer verifies the documents and sends the approval/rejection decision back to the System. The System then forwards this decision to the Admin (Object4) to update the official application status. Finally, the System notifies the Applicant about the result (approved or rejected). This clear, step-by-step message exchange ensures transparent communication, proper verification, and timely status updates among applicant, officer, system, and admin.

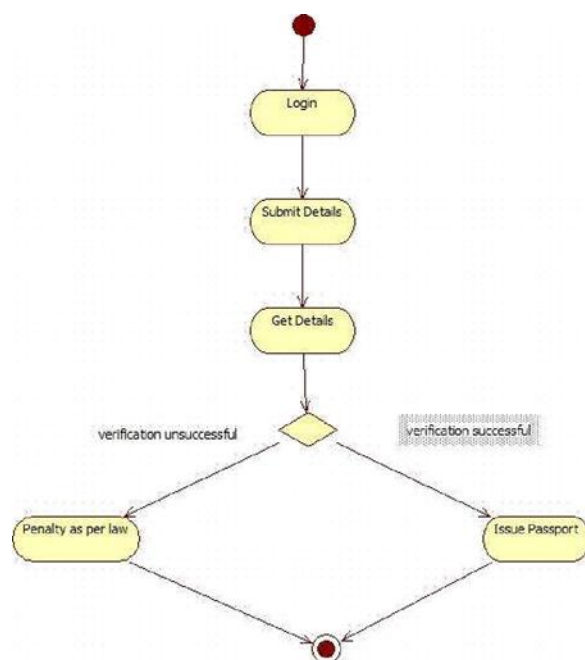
Advanced Sequence Diagram



The sequence diagram depicts the complete passport application verification process. The Applicant first logs in and submits personal details to the Passport Administrator. The Administrator verifies the details, stores them in the Database, and forwards them to the Regional Administration. The Regional Administration updates the Database and sends the details to the Police for background verification. After the Police verify and return the report, the Regional Administration updates the Database again and notifies the Passport Administrator. This multi-level verification involving local, regional, and police authorities ensures thorough scrutiny of applicant information before final processing.

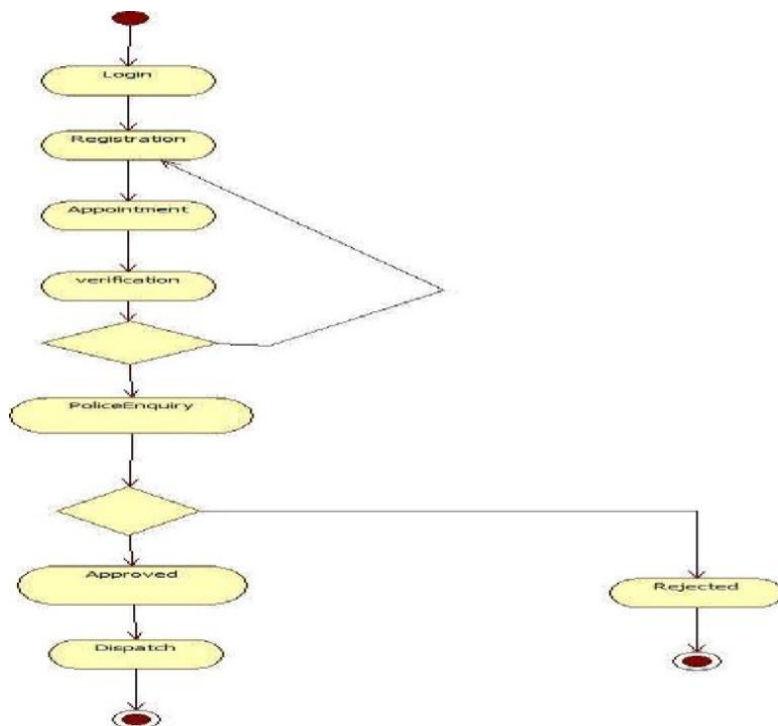
Once police verification is complete, the Passport Administrator receives confirmation, updates the Database, and the Regional Administration issues the final approval. The Database is updated with the final status, and the Passport Administrator issues the passport to the Applicant. Throughout the flow, the Database serves as the central repository, updated at every stage (initial submission, regional processing, police verification, and final issuance). This structured, step-by-step interaction among Applicant, Passport Administrator, Regional Administration, Police, and Database ensures transparency, accountability, and secure passport issuance with proper background checks and official validations at each level.

Simple Activity Diagram



This activity diagram illustrates the passport renewal/issuance process under strict verification. The applicant begins by logging in, submits personal and supporting details, and the system retrieves and verifies the stored information. A decision node checks verification outcome: if successful, the passport is issued immediately; if unsuccessful (due to discrepancies, criminal record, or invalid documents), a penalty is imposed as per law. The simple, linear flow with a single critical decision point emphasizes automated background checks and legal compliance, ensuring only eligible applicants receive passports while enforcing penalties for fraudulent or ineligible cases.

Advanced Activity Diagram



The activity diagram outlines the complete passport application lifecycle. It begins with user Login followed by Registration of personal details. The applicant then books an Appointment for document verification, after which physical Verification takes place at the passport office. A crucial decision point follows verification: if documents and identity are valid, the process continues; otherwise, it may loop back or terminate. The flow then proceeds to Police Enquiry for background and criminal record checks, ensuring eligibility. This structured sequence guarantees that only verified applicants advance to the final approval stage.

After police enquiry, another decision node determines the outcome: if clearance is received, the application is Approved and proceeds to Dispatch of the passport; if any issue is found, the application is Rejected and the process ends. The diagram clearly shows two termination points — one for successful dispatch and one for rejection. By incorporating appointment scheduling, multi-level verification (document + police), and explicit decision points, the flow ensures security, transparency, and compliance with legal procedures, preventing issuance to ineligible persons while providing a smooth path for genuine applicants from registration to final delivery.